NATIONAL AND KAPODISTRIAN UNIVERSITY OF ATHENS

SCHOOL OF SCIENCES
Department of Informatics and Telecommunications

MSc in Data Science and Information Technologies

MSc THESIS

Modular LLM Pipeline for Task-Oriented Customer Service Dialogue

Georgios Xydias

Supervisor: Kosmas Kritsis, Scientific Associate, Institute for Language and Speech Processing, Athena Research Center

ATHENS

June 2026

ΕΘΝΙΚΟ ΚΑΙ ΚΑΠΟΔΙΣΤΡΙΑΚΟ ΠΑΝΕΠΙΣΤΗΜΙΟ ΑΘΗΝΩΝ

ΣΧΟΛΗ ΘΕΤΙΚΩΝ ΕΠΙΣΤΗΜΩΝ
ΤΜΗΜΑ ΠΛΗΡΟΦΟΡΙΚΗΣ ΚΑΙ ΤΗΛΕΠΙΚΟΙΝΩΝΙΩΝ

ΜΕΤΑΠΤΥΧΙΑΚΟ ΣΤΙΣ ΕΠΙΣΤΗΜΕΣ ΔΕΔΟΜΕΝΩΝ ΚΑΙ ΤΕΧΝΟΛΟΓΙΕΣ ΠΛΗΡΟΦΟΡΙΑΣ

ΔΙΠΛΩΜΑΤΙΚΗ ΕΡΓΑΣΙΑ

Σύστημα Μεγάλων Γλωσσικών Μοντέλων (LLMs) για Διαλόγους Εξυπηρέτησης Πελατών σε Συγκεκριμένες Εργασίες

Γεώργιος Ξυδιάς

Επιβλέπων: Κοσμάς Κρίτσης, Επιστημονικός Συνεργάτης, Ινστιτούτο Επεξεργασίας του Λόγου, Ερευνητικό Κέντρο Αθηνά

ΑΘΗΝΑ

ΙΟΥΝΙΟΣ 2026

MSc THESIS

Modular LLM Pipeline for Task-Oriented Customer Service Dialogue

Georgios Xydias
S.N.: 7115152400007

SUPERVISOR: Kosmas Kritsis, Scientific Associate, Institute for Language and Speech Processing, Athena Research Center

EXAMINATION COMMITTEE:

Kosmas Kritsis, Scientific Associate, Institute for Language and Speech Processing, Athena Research Center

Vasilis Katsouros, Director of the Institute for Language and Speech Processing, Athena Research Center

Aggelos Pikrakis, Associate Professor, Department of Informatics, University of Piraeus

Examination Date: 15 June, 2026

ΔΙΠΛΩΜΑΤΙΚΗ ΕΡΓΑΣΙΑ

Σύστημα Μεγάλων Γλωσσικών Μοντέλων (LLMs) για Διαλόγους Εξυπηρέτησης Πελατών σε Συγκεκριμένες Εργασίες

Γεώργιος Ξυδιάς
Α.Μ.: 7115152400007

ΕΠΙΒΛΕΠΩΝ: Κοσμάς Κρίτσης, Επιστημονικός Συνεργάτης, Ινστιτούτο Επεξεργασίας του Λόγου, Ερευνητικό Κέντρο Αθηνά

ΕΞΕΤΑΣΤΙΚΗ ΕΠΙΤΡΟΠΗ:

Κοσμάς Κρίτσης, Επιστημονικός Συνεργάτης, Ινστιτούτο Επεξεργασίας του Λόγου, Ερευνητικό Κέντρο Αθηνά

Βασίλης Κατσούρος, Επικεφαλής Έρευνας, Ινστιτούτο Επεξεργασίας του Λόγου, Ερευνητικό Κέντρο Αθηνά

Άγγελος Πικράκης, Αναπληρωτής Καθηγητής, Τμήμα Πληροφορικής, Πανεπιστήμιο Πειραιώς

Ημερομηνία Εξέτασης: 15 Ιουνίου 2026

ABSTRACT

Task-oriented dialogue (TOD) systems are a cornerstone of modern customer-service automation, promising to handle structured user goals such as searching, booking, and information requests through natural conversation. The recent rise of instruction-tuned Large Language Models (LLMs) has reopened a fundamental design question: should a single general-purpose LLM handle the entire turn, or should the responsibilities of a dialogue agent be split into a modular pipeline of smaller, role-specific components? Existing work explores both ends of this spectrum, but rarely compares them head-to-head under a setup that uses the same dataset, the same evaluation procedure, and the same model families across architectures, leaving the practical trade-offs between architectural simplicity and architectural decomposition never brought together under a single controlled comparison.

In this work, we introduce a controlled comparative study of three task-oriented dialogue architectures for hotel and restaurant customer service, evaluated on the MultiWOZ 2.2 benchmark. The first architecture is a single-LLM baseline that performs dialogue state tracking, database lookup, and response generation in one prompt. The second is a zero-shot modular pipeline that separates the dialogue state tracker from the response generator, connected by a deterministic database lookup and a policy gate, with a lightweight guardrail on the generated response. The third applies role-specific LoRA fine-tuning to the two LLM-based modules of the pipeline (the dialogue state tracker, DST, and the response generator, ResponseGen) by training small open-source models for their assigned role via QLoRA. All three architectures are evaluated with the standardized MultiWOZ evaluator, reporting both dialogue-state metrics (Joint Goal Accuracy, Slot F1) and end-to-end metrics (Inform, Success, BLEU, Combined), complemented by a custom set of operational metrics covering hallucination, policy violations, system correctness, latency, and cost.

Our findings show that a) architectural decomposition is not universally beneficial: it substantially helps weaker models that struggle to manage every responsibility in a single prompt, but it can hurt the strongest open-source models that already coordinate all responsibilities reliably; and b) role-specific LoRA fine-tuning lifts dialogue-state tracking of small open-source models close to the level of much larger commercial APIs, making them free, fast, and competitive alternative for slot tracking. These results show that architectural decomposition and role-specific fine-tuning are effective tools, but only under the right conditions, for building reliable LLM-based customer-service dialogue systems.

SUBJECT AREA: Natural Language Processing, Task-Oriented Dialogue Systems

KEYWORDS: Large Language Models, Modular Architecture, LoRA Fine-Tuning, MultiWOZ, Customer Service Dialogue

ΠΕΡΙΛΗΨΗ

Τα συστήματα διαλόγου συγκεκριμένης εργασίας (Task-Oriented Dialogue, TOD) αποτελούν θεμέλιο λίθο της σύγχρονης αυτοματοποίησης στην εξυπηρέτηση πελατών, υποσχόμενα να διαχειρίζονται στόχους χρηστών, όπως η αναζήτηση, η κράτηση και η αίτηση πληροφοριών, μέσω φυσικής συνομιλίας. Η πρόσφατη άνοδος των Μεγάλων Γλωσσικών Μοντέλων (LLMs) που έχουν προσαρμοστεί ώστε να ακολουθούν οδηγίες, έχει επαναφέρει στο προσκήνιο ένα θεμελιώδες ερώτημα σχεδιασμού: θα πρέπει ένα μοναδικό LLM γενικού σκοπού να αναλαμβάνει ολόκληρο τον κύκλο του διαλόγου, ή θα πρέπει οι αρμοδιότητες ενός πράκτορα διαλόγου να διαμοιράζονται σε μια αρχιτεκτονική από μικρότερα, εξειδικευμένα κατά ρόλο υποσυστήματα; Η υπάρχουσα βιβλιογραφία εξερευνά και τις δυο περιπτώσεις, αλλά σπάνια τις συγκρίνει άμεσα μέσα σε ένα πλαίσιο με κοινό σύνολο δεδομένων, διαδικασία αξιολόγησης και οικογένειες μοντέλων, αφήνοντας έτσι την άμεση σύγκριση ανάμεσα στην αρχιτεκτονική απλότητα και την αρχιτεκτονική αποσύνθεση να μην έχει συγκεντρωθεί ποτέ σε μία ενιαία ελεγχόμενη μελέτη.

Στην παρούσα διπλωματική εργασία, εισάγουμε μια συγκριτική μελέτη τριών αρχιτεκτονικών διαλόγου συγκεκριμένης εργασίας για την εξυπηρέτηση πελατών στους τομείς ξενοδοχείων και εστιατορίων, η οποία αξιολογείται πάνω στο MultiWOZ 2.2. Η πρώτη αρχιτεκτονική βασίζεται σε ένα μοναδικό LLM που εκτελεί την παρακολούθηση της κατάστασης διαλόγου, την αναζήτηση στη βάση δεδομένων και την παραγωγή απάντησης μέσα σε ένα prompt. Η δεύτερη είναι μια zero-shot αρχιτεκτονική που διαχωρίζει την παρακολούθηση κατάστασης διαλόγου από τον παραγωγή απάντησης, συνδεδεμένες μεταξύ τους μέσω μιας ντετερμινιστικής αναζήτησης βάσης δεδομένων, ελέγχου πολιτικής και μιας δικλείδας ασφαλείας. Η τρίτη εφαρμόζει εξειδικευμένο κατά ρόλο fine-tuning με LoRA στα δύο LLM-based στοιχεία της αρχιτεκτονικής (DST και ResponseGen), εκπαιδεύοντας μικρά open-source μοντέλα για τον ρόλο που τους έχει ανατεθεί μέσω QLoRA. Και οι τρεις αρχιτεκτονικές αξιολογούνται με βάση το MultiWOZ, αναφέροντας τόσο μετρικές κατάστασης διαλόγου (Joint Goal Accuracy, Slot F1) όσο και μετρικές end-to-end (Inform, Success, BLEU, Combined), και επιπλέον μετρικές όπως hallucination, policy violations, system correctness, latency και cost.

Τα ευρήματά μας δείχνουν ότι α) η αρχιτεκτονική αποσύνθεση δεν είναι καθολικά ωφέλιμη: βοηθά σημαντικά τα ασθενέστερα μοντέλα, αλλά μπορεί να βλάψει τα ισχυρότερα open-source μοντέλα που ήδη συντονίζουν αξιόπιστα όλες τις αρμοδιότητες και β) το εξειδικευμένο κατά ρόλο fine-tuning με LoRA φέρνει την παρακολούθηση κατάστασης διαλόγου των μικρών open-source μοντέλων κοντά στο επίπεδο πολύ μεγαλύτερων εμπορικών APIs, καθιστώντας τα μια δωρεάν, γρήγορη και ανταγωνιστική εναλλακτική. Τα αποτελέσματα αυτά δείχνουν ότι η αρχιτεκτονική αποσύνθεση και το εξειδικευμένο κατά ρόλο fine-tuning αποτελούν αποτελεσματικά εργαλεία, αλλά μόνο υπό τις κατάλληλες συνθήκες, για την κατασκευή αξιόπιστων συστημάτων διαλόγου εξυπηρέτησης πελατών βασισμένων σε LLM.

ΘΕΜΑΤΙΚΗ ΠΕΡΙΟΧΗ: Επεξεργασία Φυσικής Γλώσσας, Συστήματα Διαλόγου Συγκεκριμένης Εργασίας

ΛΕΞΕΙΣ ΚΛΕΙΔΙΑ: Μεγάλα Γλωσσικά Μοντέλα, Νευρωνικά Δίκτυα (LLMs), Αρθρωτή Αρχιτεκτονική, Fine-Tuning με LoRA, Διάλογος Εξυπηρέτησης Πελατών

ACKNOWLEDGEMENTS

The current document is my thesis for my Master's Degree in the postgraduate program "Data Science and Information Technologies" offered by the Department of Informatics and Telecommunications of the National and Kapodistrian University of Athens.

I would like to express my sincere thanks to the supervisor of this work, Kosmas Kritsis, Scientific Associate at the Institute for Language and Speech Processing of the Athena Research Center. He gave me the opportunity to carry out the experimental part of this work on the Leonardo Booster supercomputer at EuroHPC, a high-performance cluster equipped with NVIDIA A100 GPUs. Without access to this infrastructure, the fine-tuning experiments at the core of this work would not have been possible.

This work is dedicated to my grandfather, Ioannis Vasilopoulos.

Contents

LIST OF FIGURES

LIST OF TABLES

1. Introduction

Customer service is one of the most prominent application areas of modern natural language technology, with a growing portion of everyday customer interactions such as bookings, cancellations, modifications, and information requests, handled by automated agents on behalf of businesses. Within Natural Language Processing, this kind of structured, goal-driven conversation is known as *Task-Oriented Dialogue* (TOD) [1], a setting in which a system must understand a user's request, track its evolving state across multiple turns, consult a domain database, and generate a response that moves the conversation toward the user's goal. Over the past few years, TOD has become both a thriving research area and a serious commercial concern, and the progress of the field is increasingly driven by the capabilities of Large Language Models (LLMs).

A typical task-oriented exchange illustrates why this is a non-trivial problem. Consider a user who writes: *"I'd like to book a moderately priced Italian restaurant in the centre of town for two people on Friday."* A TOD system must determine the active domain (restaurant), extract the user's constraints (food = Italian, price = moderate, area = centre, day = Friday, people = 2), query the restaurant database for matching entities, decide whether enough information has been gathered to commit a booking, and finally produce a fluent response that either confirms the booking, asks for missing details, or proposes an alternative when no match exists. Each of these sub-tasks is well-studied in isolation. The question is how to best combine them inside a modern LLM-based agent.

Two largely independent design choices shape any LLM-based TOD system. The first is *decomposition*: whether a single LLM owns the entire turn (monolithic approaches, which feed the dialogue history and the necessary database information into one prompt and produce the next system response in one pass [3, 10]) or whether the turn is split into role-specific components such as a dialogue state tracker, a database lookup step, and a response generator, connected by deterministic intermediate steps (modular approaches [10, 11, 14, 16]). The monolithic side offers simplicity, with no intermediate steps to define and little engineering effort required; the modular side offers interpretability and control, since each module can be inspected, replaced, or trained independently. The second choice is *specialization*: whether the LLM components involved are general-purpose instruction-tuned models, or have been adapted to a specific dialogue role through parameter-efficient fine-tuning techniques such as LoRA [19] and QLoRA [20], which enable small open-source LLMs to be specialized at a fraction of the cost of full fine-tuning [21]. These two axes are conceptually independent: a system can decompose without specializing, specialize without decomposing, or do both.

Despite the volume of work on each of these designs individually, a head-to-head comparison among all three under controlled conditions is largely missing from the literature. Existing studies typically evaluate one design at a time, often on different datasets, with different evaluation tools, and with different model families, which makes the reported numbers difficult to compare across studies. A practitioner who is choosing between a single-prompt agent, a zero-shot modular pipeline, and a LoRA-fine-tuned modular pipeline therefore has access to many promising data points but very few apples-to-apples comparisons. As a result, the practical trade-offs between architectural simplicity and architectural decomposition, and the conditions under which each design pays off have never been brought together in one study.

This work introduces a Modular LLM Pipeline for Customer Service, a controlled comparative study that addresses exactly this gap. We implement and evaluate three task-oriented dialogue architectures for hotel and restaurant customer service on the MultiWOZ 2.2 benchmark [2], using the same dataset, the same standardized evaluator [23], and the same families of commercial and open-source models across all three architectures. The first architecture is a monolithic single-LLM baseline that performs dialogue state tracking, database lookup, and response generation within a single prompt. The second is a zero-shot modular pipeline that separates dialogue state tracking from response generation, connected by a deterministic database lookup, a policy gate that prevents invalid bookings, and a lightweight guardrail on the generated response. The third applies role-specific LoRA fine-tuning, via QLoRA, to the two LLM-based modules of the modular pipeline (the dialogue state tracker and the response generator), training small open-source models for their assigned role.

Across multiple commercial and open-source LLMs of varying sizes, our findings reveal a consistent pattern. Architectural decomposition is conditional rather than universal: it substantially helps weaker models that struggle to manage every responsibility within a single prompt, but it can hurt the strongest open-source models that already coordinate all responsibilities reliably. Role-specific LoRA fine-tuning, in turn, lifts the dialogue state tracking of small open-source models close to the level of much larger commercial APIs, opening a path to free, fast modular agents that match commercial slot trackers in their core extractive task without their cost. Taken together, the results suggest that decomposition and role-specific fine-tuning should be applied selectively, based on the strength of the base model and the specific role being targeted, rather than as universal recipes for reliable LLM-based customer-service dialogue.

In summary, this work contributes the following. We introduce a controlled experimental framework that holds dataset, evaluator, and model families constant across three families of LLM-based task-oriented dialogue architectures, providing the apples-to-apples comparison that the literature currently lacks. We design and implement a zero-shot modular pipeline that combines two LLM modules (a dialogue state tracker and a response generator) with a deterministic database lookup, a policy gate, and a lightweight guardrail on the generated response. We extend this pipeline with role-specific LoRA adapters trained via QLoRA on small open-source backbones, demonstrating a practical recipe for free, fast, role-specialized modular agents. Finally, we conduct an extensive empirical study, accompanied by both standard MultiWOZ metrics and a custom set of operational metrics, that maps architectural and training choices to their measurable impact on dialogue state tracking, response quality, and operational behavior on MultiWOZ 2.2.

2. Related Work

Research on task-oriented dialogue dates back more than a decade, but the character of the field has shifted dramatically with the arrival of large language models. The pre-LLM era was dominated by single, end-to-end neural models fine-tuned on task-specific data. SimpleTOD [3] is the classic example: a pretrained GPT-2 backbone fine-tuned on MultiWOZ to produce belief state, system action, and response in a single autoregressive sequence. Its philosophy, *one model, all responsibilities*, established the monolithic design paradigm which this work adopts as its baseline.

The arrival of GPT-3 [4] and InstructGPT [5] shifted attention toward zero-shot and few-shot approaches that require no task-specific training. Hudeček and Dušek [10] tested this premise directly on MultiWOZ, evaluating two zero-shot architectures with GPT-3.5: a single-prompt setup that handles the entire turn at once, and a modular pipeline that separates domain detection, dialogue state tracking, database lookup, and response generation. Their work is the standard zero-shot LLM baseline for the field and the direct counterpart of the first two architectures in this work. Several follow-up studies refined the prompts themselves rather than the architecture: SGP-TOD [11] grounds each prompt with the dataset's schema and a handwritten task flow, while Hu et al. [12] showed that retrieving similar training dialogues and injecting them as few-shot examples substantially improves dialogue state tracking. Together these works established that prompt engineering alone can take an off-the-shelf LLM a long way on MultiWOZ, but they did not address whether decomposition into specialized modules is itself helpful.

A parallel line of work has reframed TOD as a multi-agent or agentic problem. Xu et al. [13] propose a zero-shot autonomous agent that replaces traditional modularity with an LLM-driven reasoning loop. DARD [14] organizes the agent as a team of specialized sub-agents that coordinate to handle a turn. Baidya et al. [15] benchmark several zero-shot LLM agents against complex MultiWOZ dialogues and document a sizable behavior gap between agentic systems and traditional pipelines. More recently, Feng et al. [16] combine a multi-agent framework with role-specific fine-tuning, demonstrating that the two ideas of decomposition and parameter efficient training can be combined productively. These works are spiritually aligned with this work but evaluate their contributions in isolation, on different datasets and against different baselines, leaving direct architecture-versus-architecture comparison out of scope.

Parameter-efficient fine-tuning is the third major thread relevant to this work. LoRA [19] introduced the now-standard approach of training small low-rank adapters on top of a frozen pretrained model, and QLoRA [20] extended it with 4-bit quantization, making it possible to fine-tune large LLMs on a single consumer or academic GPU. Spec-TOD [21] is the most direct cousin of the Experiment 3 of this work: it fine-tunes LLaMA-3-8B with LoRA on hand-crafted TOD instructions and reaches competitive performance on MultiWOZ 2.2 using only a fraction of the training data. Spec-TOD trains a single end-to-end model across all seven MultiWOZ domains, however, whereas this work trains two separate role-specific adapters, one for dialogue state tracking, one for response generation, restricted to the hotel and restaurant domains, allowing the contribution of role specialization itself to be measured directly.

Two further lines of work shape this work: one provides the vocabulary used to describe its architectures, and the other provides the evaluation tools used to measure them. Kranti et al.

[17] introduced *clem:todd*, a benchmarking framework that proposes a clean vocabulary for LLM-based TOD architectures: *monolithic* systems where one LLM does everything, *modular-programmatic* systems where handwritten code orchestrates LLM calls, and *modular-LLM* systems where multiple LLM components each own a role. This work adopts that vocabulary directly, with Experiment 1 corresponding to the monolithic class and Experiments 2 and 3 to the modular-LLM class. On the evaluation side, Deriu et al. [22] survey methodologies for dialogue system evaluation and ground the choice of metrics used here, while Nekvinda and Dušek [23] released the standardized mwzeval evaluator that this work uses to compute Inform, Success, BLEU, and Combined, eliminating the inconsistencies that plagued earlier MultiWOZ comparisons.

Across all of these directions, the literature offers many promising data points but few direct comparisons. Studies of zero-shot LLM TOD typically use GPT-3.5 alone; studies of fine-tuned modular pipelines typically use a single open-source family; agentic frameworks typically evaluate themselves against a handpicked baseline rather than across designs. The trade-offs between architectural simplicity and architectural decomposition, and between zero-shot prompting and role-specific fine-tuning, therefore remain difficult to assess from the published record. This work addresses that gap by holding the dataset, the evaluator, and the model families constant across three architectures, a single-LLM baseline, a zero-shot modular pipeline, and a LoRA-fine-tuned modular pipeline, and comparing them under a single experimental protocol on MultiWOZ 2.2.

3. The Task-Oriented Dialogue Problem

A dialogue is a sequence of turns alternating between a user and a system, where each turn is a single utterance. At each user turn *t*, a task-oriented dialogue (TOD) system [1] is given the dialogue history (all previous turns) together with a domain-specific database, and is required to produce the next system response. The response should move the user toward their goal, whether that goal is searching for an entity, requesting information about it, or completing a transaction such as a booking, and it should remain faithful to the contents of the database and consistent with all of the constraints the user has expressed across earlier turns.

Most TOD systems, regardless of architecture, perform five canonical sub-tasks at each turn, either explicitly as named modules or implicitly inside a single forward pass. *Domain detection* identifies which domain the current turn concerns, since the same dialogue can span multiple domains and each domain has its own slot schema and database. *Intent detection* identifies the user's communicative goal at the current turn. Most commonly, whether the user is still searching for an entity or has committed to booking one, since the intent determines which downstream actions are appropriate. *Dialogue state tracking* (DST) maintains a structured belief state: a set of slot-value pairs that captures all of the user's constraints accumulated across turns. The belief state is cumulative as new constraints expressed at turn *t* are added to those carried over from earlier turns, and a value remains in effect until it is overridden later in the dialogue. *Database lookup* uses the current belief state as a query against the domain database, returning the set of entities consistent with the constraints; for transactional intents, it also commits a booking once all required slots are filled. *Response generation* produces the final system utterance, conditioned on the dialogue history and on whatever database evidence the system has available. For evaluation purposes the response is typically considered in two forms: a *delexicalized* form, in which entity-specific values are replaced by generic placeholders such as [restaurant_name] or [hotel_phone], and a *lexicalized* form, in which the placeholders have been filled in with the actual values returned from the database. The delexicalized form is the standard target of end-to-end metrics; the lexicalized form is what the user sees.

These five sub-tasks are well-defined as conceptual ingredients, but they leave the central architectural question open: which of them are exposed as separate components, and which are absorbed into a single LLM call? At one extreme, all five can be folded into a single prompt that asks the model to produce a response in one pass, with belief state and database evidence handled internally and never surfaced. At the other extreme, each sub-task can become its own module with its own prompt and intermediate output, connected by deterministic code. This choice, together with the parameter-efficient specialization of individual modules on top of it, is the central object of study of this work, and is mapped systematically in the next chapter.

The scope of this work is closed-domain, slot-filling task-oriented dialogue grounded in a single database per domain. Concretely, all three architectures are implemented and evaluated on the hotel and restaurant domains of the MultiWOZ 2.2 benchmark [2], each domain being a self-contained database of entities described by a fixed slot schema. Open-domain dialogue, social conversation, multi-database reasoning that requires joining across heterogeneous sources, and speech input or output are out of scope. The dataset is text-only and in English, and the evaluation considers a single user goal per dialogue at a time.

4. Architectures for LLM-Based Task-Oriented Dialogue

Architectural choices for LLM-based task-oriented dialogue can be organized along two largely independent axes. The first axis is *decomposition*: how the canonical sub-tasks identified in the previous chapter are distributed across LLM calls and deterministic code. The second is *specialization*: whether the LLM components involved are general-purpose instruction-tuned models or role-specialized models obtained through parameter-efficient fine-tuning. These two axes are conceptually independent. A system can decompose without specializing, specialize without decomposing, or do both. The three architectures evaluated in this work correspond to three distinct points in the space they define.

Along the decomposition axis, the *clem:todd* benchmarking framework of Kranti et al. [17] proposes a clean three-way classification that this work adopts directly. A *monolithic* system is one in which a single LLM owns the entire turn: dialogue history and any necessary database evidence enter a single prompt, and the system response emerges in one forward pass. Belief state, database lookup, and response generation may all be performed internally by the model. A *modular-programmatic* system splits the turn into multiple LLM calls connected by handwritten code, in which the deterministic glue between calls, such as domain routing, database queries, slot reconciliation, is implemented explicitly rather than left to the model's own reasoning. A *modular-LLM* system goes one step further: it dedicates a distinct LLM component to each of one or more canonical sub-tasks, exposes their intermediate outputs, and treats the belief state and the response as separate, named artifacts that flow through the pipeline. Experiment 1 of this work belongs to the monolithic class; Experiments 2 and 3 belong to the modular-LLM class.

The trade-offs along this axis motivate the comparison the work runs. Monolithic designs are simple to implement, require very little orchestration code, and place no constraints on the format of the model's internal reasoning. Their drawbacks are equally direct. A monolithic system has only one prompt to debug and only one point of intervention; when it fails, it is rarely clear whether the failure originated in slot extraction, in database reasoning, or in response generation. Long dialogue histories combined with a full database render of even a single domain can quickly exhaust the context window of small open-source models, and any error in any part of the response is paid for in the same forward pass. Modular designs invert this picture. Each module can be inspected in isolation, replaced, prompt-engineered, or fine-tuned independently, and the responsibilities are partitioned so that a single role-specific prompt can be much shorter and more focused than a monolithic one. The cost is the additional engineering surface of multiple prompts, intermediate representations, deterministic connectors, and the risk of cascading errors, in which a mistake in an upstream module is faithfully propagated perfectly to a downstream one.

The second axis cuts across the first. *Specialization* refers to the distinction between general-purpose instruction-tuned LLMs, whether commercial APIs or open-source, and models that have been fine-tuned for a specific dialogue role. Parameter-efficient fine-tuning has made the specialized end of this axis broadly accessible: LoRA [19] introduced low-rank adapters that train only a small fraction of a model's parameters while keeping the original weights frozen, and QLoRA [20] extended the technique with 4-bit quantization, enabling adapters to be trained on top of large base models with the memory budget of a single academic GPU. Specialization is orthogonal to decomposition. A monolithic system can be fine-tuned end-to-end, as in Spec-

TOD [21], which trains a single LoRA adapter to handle all canonical sub-tasks across all MultiWOZ domains. A modular-LLM system can leave its modules zero-shot and rely entirely on prompt engineering, as in Hudeček and Dušek [10]. Or, as in this work, a modular-LLM system can apply role-specific LoRA adapters to its individual modules, training each adapter only for the canonical sub-task its module is responsible for.

A working modular pipeline is rarely composed of canonical-sub-task modules alone. In practice it also contains deterministic *engineering scaffolding* that holds the system together: a policy gate that enforces task-level invariants such as not committing a booking before all required slots are filled; a supervisor or guardrail that validates the generated response against the current belief state and database evidence and, if necessary, triggers a bounded number of retries; a lexicalizer that fills delexicalized placeholders with actual database values; and a turn-level memory that decides what is appended to the dialogue history for subsequent turns. None of these components belong to the canonical sub-tasks of the previous chapter, and none of them by themselves change the architectural class of a system in the *clem:todd* sense. They are mentioned here only to disclose where they sit conceptually; their concrete role inside this work's pipeline is the subject of the next chapter.

The three architectures evaluated in this work can now be placed onto the resulting two-axis map. Experiment 1 is *monolithic*, with no specialization: a single general-purpose LLM, commercial or open-source, performs the entire turn within one prompt. Experiment 2 is *modular-LLM*, again without specialization: dialogue state tracking and response generation are separate LLM modules connected by deterministic database lookup, policy, supervisor, lexicalizer, and memory components, but every LLM module is a general-purpose model used zero-shot. Experiment 3 keeps the modular-LLM structure of Experiment 2 unchanged and applies role-specific LoRA fine-tuning, via QLoRA, to each of its two LLM modules, one adapter for the dialogue state tracker and a separate adapter for the response generator, without touching the deterministic scaffolding around them. With this map in place, the next chapter can describe each of the three architectures in concrete detail.

5. System Architecture

The previous chapter mapped the design space for LLM-based task-oriented dialogue along two axes: decomposition and specialization, placing the three architectures evaluated in this work at three distinct points within it. This chapter describes each of those three architectures in concrete operational detail: which inputs each component receives, which artifacts it produces, and how a single user turn is processed end to end. The deterministic scaffolding is described once, in the context of Experiment 2, and then noted as unchanged for Experiment 3.

The first architecture, **Experiment 1**, is a monolithic system in which every responsibility of a user turn is folded into a single LLM call. The system prompt contains both target domain databases, hotels and restaurants, serialized in full, together with the assistant's role description. The user prompt then carries the dialogue history so far, the current belief state, the schema of valid domains, intents and slots, the booking-policy requirements, and a short set of extraction and response rules. The model is asked to return a single JSON object containing four fields: the active domain, the active intent, the slot-value updates extracted from the user's latest turn, and the delexicalized response itself. Although these four fields are surfaced as named outputs, they are produced by one forward pass with no intermediate intervention: there is no separate dialogue state tracker invocation, no separate response-generator invocation, and nothing that conditions one of these outputs on another beyond what the model performs internally. After the call returns, the system merges the newly extracted slots into the accumulated belief state and then performs a deterministic database lookup whose sole purpose is *lexicalization*, replacing placeholders such as [hotel_name], [hotel_phone] and [ref] in the model's delexicalized response with the corresponding values from the database. There is no retry mechanism and no policy gate that influences what the model produces; the policy and supervisor checks are still computed afterwards, but only for the evaluation metrics, since the response itself is fixed by the single LLM call. A direct practical consequence of this design is that prompts grow substantially with every turn, because both domain databases and the cumulative dialogue history are re-rendered on each call. As the dialogue progresses, the prompt commonly exceeds twenty thousand tokens, which means Experiment 1 can only be run reliably on models with very large context windows, in practice the commercial APIs and the largest open-source models; for small open-source models, it’s difficult to handle. This context-window pressure is itself one of the empirical motivations for the modular pipelines that follow.

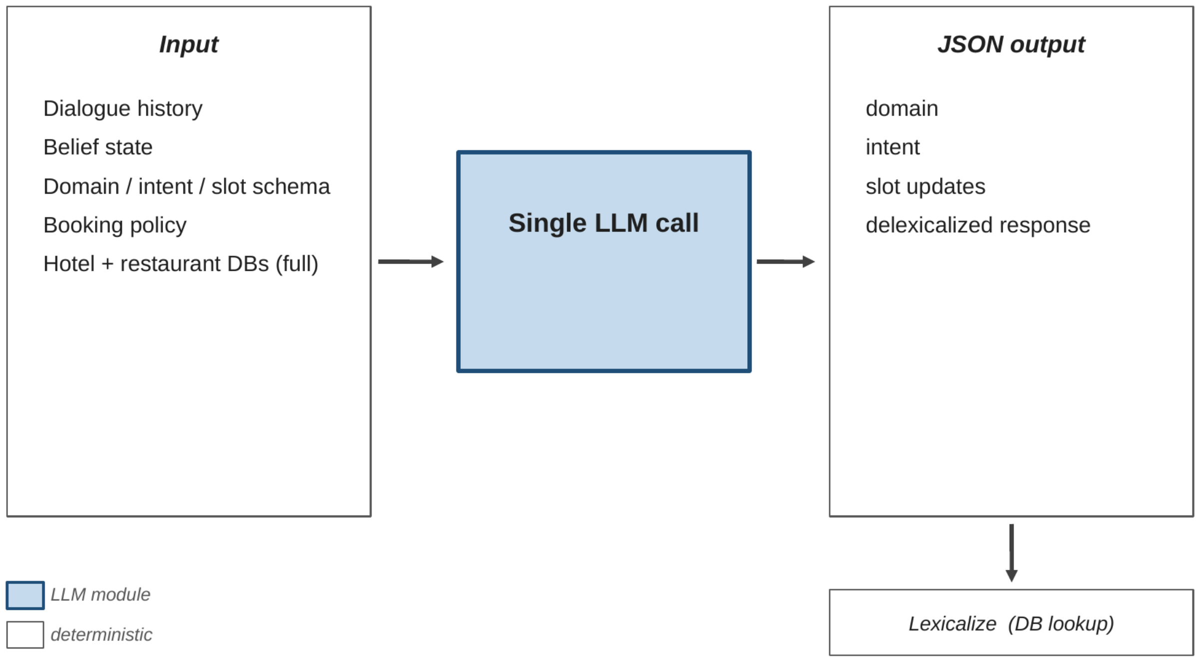

Figure 5.1: Per-turn data flow of Experiment 1 (monolithic single-LLM architecture).

The full hotel and restaurant databases enter the prompt together with the dialogue history, the current belief state, the schema, and the booking policy; a single LLM call produces a JSON object with the active domain, intent, slot updates, and delexicalized response. The downstream lexicalization step is deterministic and uses a database lookup only to fill placeholders.

The second architecture, **Experiment 2**, replaces the single-prompt design with an explicitly modular pipeline composed of two LLM modules and four deterministic components, all orchestrated by a fixed per-turn procedure. At each user turn the pipeline first calls the *dialogue state tracker* (DST) module, which receives the dialogue history together with the user's latest utterance and returns three named artifacts: the active domain, the active intent, and the *accumulated belief state*, a slot-value dictionary that grows monotonically across turns and carries the full set of constraints the user has expressed so far. The accumulated belief state and the intent are then passed to the deterministic *policy* component, which inspects the current intent against a fixed table of booking requirements, for example, a book_hotel intent requires the slots *day*, *stay* and *people*. The policy returns a list of *violations*: slots that the current intent requires but that the belief state does not yet contain. The pipeline then performs the *database lookup*: for search intents it queries the relevant domain database for entities consistent with the constraints in the belief state, and for booking intents with no policy violations it commits a booking by writing back into the database, treating the booking attempt itself as the lookup step. The result of the lookup, a possibly empty list of matching entities, is handed together with the belief state, the intent, the violations, and the dialogue history to the *response generator* (ResponseGen) module, which produces a delexicalized response recommending a single entity at a time, the first one returned by the lookup, using a fixed inventory of placeholders such as [hotel_name], [hotel_phone] and [ref]. The response is then validated by the deterministic *supervisor*, which checks it against the current belief state, the database results, and the policy violations; if the response is judged invalid, the supervisor returns textual feedback that is fed back into a fresh ResponseGen call as additional prompt context. The supervisor can trigger at most one such retry, capping the pipeline at two response-generation attempts per turn; if the second attempt is also rejected, the last response produced is accepted regardless. Once a response has been accepted, the *lexicalizer* substitutes each placeholder with the corresponding value from the database results, producing the lexicalized response that the user actually sees. Finally, the *memory* component appends the user utterance and the lexicalized response to the dialogue history, so that subsequent turns operate on text without unresolved placeholders. Two design choices in this pipeline deserve explicit mention. First, the accumulated belief state grows turn by turn and is never reset within a dialogue, mirroring the standard MultiWOZ convention that constraints from earlier turns remain in effect until they are explicitly overridden. Second, the dialogue history stored by the memory component contains the *lexicalized* responses, not the delexicalized ones; storing placeholders would contaminate subsequent turns by re-introducing them into the DST and ResponseGen prompts.

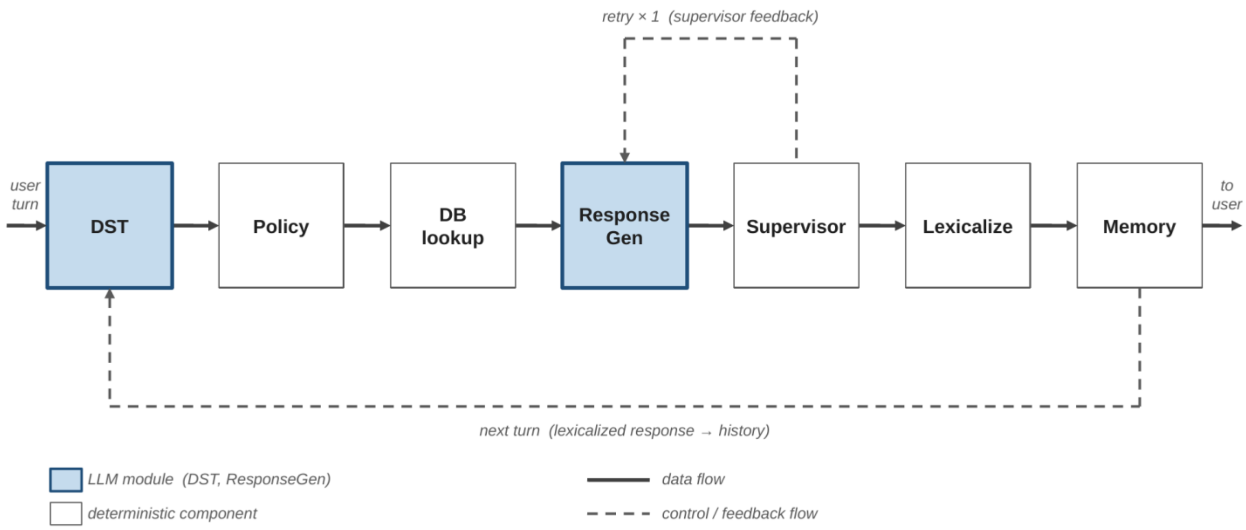

Figure 5.2: Per-turn data flow of the modular pipeline used in Experiments 2 and 3.

The two LLM modules (DST and ResponseGen, shaded) are powered by general-purpose models in Experiment 2 and by role-specific LoRA adapters in Experiment 3; the deterministic components (Policy, DB lookup, Supervisor, Lexicalize, Memory) are identical across both experiments. The supervisor can trigger up to one retry of ResponseGen with feedback as additional prompt context. At the end of each turn, the lexicalized response is appended to the dialogue history and the pipeline restarts at DST for the next user turn.

The third architecture, **Experiment 3**, keeps the entire structure of Experiment 2 unchanged. The same per-turn procedure is followed, the same deterministic components are used, the same intermediate artifacts flow between them, and the same validation-and-retry loop is applied. The difference lies entirely in what powers the two LLM modules. In Experiment 2, both DST and ResponseGen are general-purpose instruction-tuned models used zero-shot. In Experiment 3, each of these modules is replaced by a small open-source model carrying a role-specific LoRA adapter that has been trained to perform exactly the canonical sub-task assigned to it: one adapter for dialogue state tracking, a separate adapter for response generation. The deterministic scaffolding around them such as policy, database lookup, supervisor, lexicalizer, and memory, is identical to its Experiment 2 counterpart and is in fact executed by the same code path. A non-obvious but essential property of this design is *train-inference prompt* *consistency*: the same prompt-construction functions used to assemble a DST or ResponseGen prompt at inference time are also the functions that produce the training examples seen by the adapters during fine-tuning. The adapters therefore never encounter at inference a prompt format that differs from the format they saw during training, which is what makes the modular pipeline a clean substrate for parameter-efficient specialization in the first place.

With these three architectures defined operationally, the comparison the work runs is fully specified at the system level. When the same general-purpose model family is placed into Experiments 1 and 2, any observed difference in behavior is attributable to architectural decomposition alone; and when the modular structure of Experiments 2 and 3 is held constant and only the LLM modules vary, any observed difference is attributable to role-specific specialization alone. The next chapter describes the experimental setup that runs each of these architectures: which models are placed into which slots, how the LoRA adapters of Experiment 3 are trained, and how all three architectures are evaluated.

6. Experimental Setup

This chapter describes the experimental setup that runs the three architectures of this work: the dataset on which they are evaluated, the models placed into each role, the fine-tuning recipe used to specialize the two LLM modules of Experiment 3, and the metrics used to measure them. Apart from the fine-tuning recipe, which is specific to Experiment 3, the same setup is shared across all three Experiments.

6.1 Dataset

All three architectures are evaluated on the MultiWOZ 2.2 benchmark [2], the corrected and re-annotated successor to the original MultiWOZ corpus and the de facto standard dataset for task-oriented dialogue research. MultiWOZ 2.2 spans seven domains: attraction, hotel, restaurant, taxi, train, hospital, and police; each with its own slot schema and database. As stated in Chapter 3, this work restricts itself to the *hotel* and *restaurant* domains, the two domains with the richest schemas, the most balanced search-and-book intent mix, and the largest number of dialogues. To enforce the restriction at the data level, dialogues whose service set extends beyond hotel and restaurant are filtered out before evaluation; only dialogues that take place entirely within the two target domains, possibly switching from one to the other, are kept.

Evaluation is performed on the MultiWOZ 2.2 *test* split filtered to the two target domains, which contains 186 dialogues and 1,100 user turns. The training and development splits, restricted to the same two domains, contain 2,850 and 171 dialogues respectively (14,464 and 1,038 user turns), and are used only by Experiment 3: the training split provides the supervised pairs for the LoRA adapters, the development split drives validation loss and early stopping during fine-tuning, and the test split is held out from training entirely and used only to report final numbers. Across the filtered subset, dialogues average between five and six user turns each. Experiments 1 and 2 are zero-shot and therefore touch only the test split. The three architectures are evaluated on the same set of test dialogues and the same set of user turns, which is what makes the apples-to-apples comparison possible at the data level.

The slot schema relevant to evaluation comprises both *informable* slots that carry user constraints (e.g., *area*, *food*, *pricerange*, *stars*, *internet*, *parking*) and *booking* slots that the policy gate enforces (*day*, *stay*, and *people* for hotel bookings; *day*, *time*, and *people* for restaurant bookings). A user turn is excluded from per-turn metric averages only when the predicted domain and the predicted intent are both null, which corresponds in practice to multi-domain transitions and farewell turns; under this exclusion rule, slot-level metrics are still computed when applicable, but classification metrics that are undefined for a null prediction are left out of the average rather than counted as wrong predictions.

6.2 Models and configurations

The model inventory is selected to include both the access-mode axis, commercial APIs versus open weights, and the parameter-scale axis, from 3 billion to about 14 billion parameters on the open-source side. The commercial APIs evaluated are GPT-4o-mini (OpenAI, July 2024), GPT-4.1-nano (OpenAI), and Claude Haiku 4.5 (Anthropic, October 2025), chosen as the cost-effective instruction-tuned production options at the time of evaluation rather than the most capable models available. The open-source models, all loaded in 4-bit quantization through Unsloth's *bnb-4bit* builds, are LLaMA-3.2-3B-Instruct and LLaMA-3.1-8B-Instruct from Meta, Qwen2.5-14B-Instruct, Qwen3-4B, Qwen3-8B, and Qwen3-14B from Alibaba, and Phi-4-14B from Microsoft, drawn from three vendors and several architectural lineages so that the comparison spans both scale and family at each parameter level. Table 6.1 summarizes each model along the dimensions that matter for this study.

Throughout this work, models are referred to by their full name in body text (Phi-4-14B, Claude Haiku 4.5, Qwen3-14B) and by short code labels in tables and configuration names (phi4_14b, haiku, qw3_14b). The full mapping is given in Table 6.1.

Table 6.1: Models evaluated and their coverage across the three Experiments.

Model	Label	Params	Context	Access	Exp1	Exp2	Exp3
GPT-4o-mini	gpt	undisclosed	128 K	API (OpenAI)	✓	✓	–
GPT-4.1-nano	gpt-nano	undisclosed	128 K	API (OpenAI)	✓	–	–
Claude Haiku 4.5	haiku	undisclosed	200 K	API (Anthropic)	✓	✓	–
LLaMA-3.2-3B-Instruct	lla32_3b	3.2 B	128 K	open (Llama 3.2)	–	✓	✓
LLaMA-3.1-8B-Instruct	lla31_8b	8.0 B	128 K	open (Llama 3.1)	✓	✓	✓
Qwen2.5-14B-Instruct	qw25_14b	14.7 B	128 K	open (Apache 2.0)	✓	✓	–
Qwen3-4B	qw3_4b	4.0 B	32 K	open (Apache 2.0)	✓	✓	✓
Qwen3-8B	qw3_8b	8.2 B	32 K	open (Apache 2.0)	✓	✓	✓
Qwen3-14B	qw3_14b	14.8 B	32 K	open (Apache 2.0)	✓	✓	✓
Phi-4-14B	phi4_14b	14 B	16 K	open (MIT)	✓	✓	✓

Two further models, Phi-4-mini and Gemma-2-9B, were attempted but could not be loaded by the inference stack used in this work. Phi-4-mini relies on a custom long-context positional-encoding scheme not supported by the Unsloth and transformers versions used here, and Gemma-2-9B's chat template does not declare a system role, which every prompt builder in the pipeline requires. Both models are excluded from the inventory.

The configurations actually run for each Experiment are not all combinations of the inventory above with every role; they are deliberately chosen to answer specific questions. Experiment 1 tests every model that fits into the monolithic role, while Experiment 2 runs both *homogeneous* configurations, in which the same model fills the DST and ResponseGen slots, and *heterogeneous* configurations, in which the two roles carry different models. The homogeneous configurations isolate the effect of decomposition on each individual model answering *does this model improve when its responsibilities are split?* while the heterogeneous configurations probe more targeted questions: which API role matters more (DST or ResponseGen), by swapping GPT-4o-mini and Claude Haiku 4.5 across the two roles; whether a JSON-optimized backbone helps DST specifically, by combining Qwen2.5-14B as DST with Qwen3-14B as ResponseGen; and whether a strong DST can rescue a weaker ResponseGen, by pairing Qwen3-14B with

Qwen3-8B and with Phi-4-14B. Experiment 3 trains a separate LoRA adapter for each role on each enrolled open-source backbone and evaluates both homogeneous fine-tuned configurations with same backbone, two role-specific adapters and a small set of heterogeneous fine-tuned configurations that mix backbones across the two roles.

This combination of homogeneous and heterogeneous configurations across all three Experiments is the operational realization of the apples-to-apples claim the work promises. A given model of Qwen3-14B, for example, appears as the monolithic LLM in Experiment 1, as both modules of homo_qw3_14b in Experiment 2, and with role-specific adapters in ft_homo_qw3_14b in Experiment 3, all evaluated on the same 186 test dialogues with the same metrics. Any difference in the observed behavior across these three configurations is due to architectural and specialization choices alone, not to model identity.

6.3 Fine-tuning recipe

Experiment 3 is the only part of the work that involves training. For each open-source backbone enrolled in this Experiment, two independent LoRA [19] adapters are trained from the same base model, one for the dialogue state tracker, one for the response generator, and combined at inference time inside the modular pipeline of Chapter 5. Training runs in 4-bit precision via QLoRA [20], with the adapters themselves kept at 16-bit, allowing the full set of backbones up to 14 billion parameters to be fine-tuned on a single NVIDIA A100 GPU on the Leonardo Booster supercomputer at EuroHPC. The same recipe is used for both adapters and across all backbones, so that any difference in observed behavior between two fine-tuned configurations is due to the choice of backbone and not to per-run hyperparameter tuning.

Training data is constructed directly from the prompt-builder functions of the inference pipeline. Each MultiWOZ 2.2 training dialogue is replayed turn by turn against the dataset's ground-truth annotations: the same function that builds a DST prompt at inference time is invoked with the same dialogue history and current user utterance to produce the *input* of a training pair, and the corresponding ground-truth slot dictionary, serialized in the same format the inference parser expects, becomes the *target*. The same procedure is followed for the response generator, with the ground-truth delexicalized response as target. Each example is then formatted as a raw Alpaca-style record (### Instruction: ... ### Input: ... ### Response: ...) without chat templates or special reasoning tokens, and the resulting dataset is fed to a HuggingFace SFT Trainer from the TRL library. The MultiWOZ 2.2 development split, processed through the same procedure, drives evaluation loss every 100 training steps and triggers early stopping when validation loss fails to improve for three consecutive evaluations. The MultiWOZ 2.2 test split is never seen during training. This construction is the operational guarantee of the *train-inference prompt consistency* property promised in Chapter 5: each adapter is trained on exactly the prompts it will encounter at inference, character for character.

Table 6.2 lists the LoRA, QLoRA, and trainer hyperparameters used for every fine-tuning run in Experiment 3. The values follow the Unsloth/QLoRA defaults that have become standard in recent task-oriented dialogue work, including Spec-TOD [21]; the rank-16 configuration with all attention and MLP projections targeted is wide enough to capture role-specific behavior without inflating the trainable-parameter count beyond a small fraction of the base model's total. The maximum sequence length of 2,048 tokens comfortably accommodates the longest observed prompts in the training set, around 1,580 tokens for the DST adapter and 1,326 tokens for the response generator adapter.

Table 6.2: Fine-tuning hyperparameters used for all LoRA adapters trained in Experiment 3.

Parameter	Value
LoRA rank	16
LoRA alpha	16
LoRA dropout	0
Target modules	all attention and MLP projections (q, k, v, o, gate, up, down)
Base precision	4-bit NF4 (QLoRA)
Adapter precision	16-bit
Effective batch size	8 (per-device 2 × gradient accumulation 4)
Optimizer	adamw_8bit
Learning rate	2×10^{-4}
LR schedule	linear
Warmup steps	10
Weight decay	0.001
Max epochs	3 (capped, early stopping decides actual end)
Eval / save frequency	every 100 steps
Early-stopping patience	3 evaluations
Max sequence length	2,048 tokens
Random seed	3407

6.4 Evaluation metrics

Two families of metrics are reported for every configuration of every Experiment. The first family is the set of *standardized* MultiWOZ end-to-end metrics of Inform Rate, Success Rate, BLEU, and Combined, computed via the *mwzeval* evaluator of Nekvinda and Dušek [23], which has been adopted as the de facto standard scoring library for MultiWOZ since it eliminated the inconsistencies that plagued earlier implementations. Inform Rate measures whether the system recommended an entity that satisfies the user's informable constraints; Success Rate measures whether the system additionally communicated all of the slot values the user requested across the dialogue; BLEU is a corpus-level similarity score against the reference responses; and Combined is the standard composite *(Inform + Success) / 2 + BLEU*. These metrics are computed on the *delexicalized* form of the system response, mirroring the official MultiWOZ leaderboard convention. Since the present evaluation covers two of the seven MultiWOZ domains rather than all seven, the absolute numbers are not directly comparable to the published leaderboard.

The second family is a custom set of *operational* metrics designed to expose the per-component behavior that the standardized metrics aggregate away. They are defined in Table 6.3 and are computed on the *lexicalized* form of the system response, since some of them, most notably hallucination rate, only make sense once placeholders have been filled with database values. Three of these metrics correspond to per-turn classification quality: Domain Precision and Intent Precision measure whether the DST module assigns the correct domain and intent on each turn, and Action Accuracy is a composite that requires both classification outputs and the per-turn slot updates to be correct simultaneously. Three further metrics target the quality of the belief state itself: Joint Goal Accuracy is the strict full-state metric, turning a turn into a success only when every slot–value pair in the predicted accumulated belief state matches the ground truth exactly; Slot Recall and Slot F1 relax this strictness to per-slot precision and recall against the ground-truth belief state. Three further metrics target operational reliability: Hallucination Rate flags turns whose lexicalized response mentions an entity not returned by the database lookup of that turn; Policy Violation Rate flags turns where a booking intent is acted upon despite the policy gate having recorded missing required slots; and System Correctness is the composite of the previous two, requiring the response to be free of both kinds of error simultaneously. Booking Rate is the fraction of dialogues with at least one booking intent in which a booking confirmation is reached, and dialogues without any booking intent are excluded from this average. Finally, Latency per turn aggregates total wall-clock time per user turn including any supervisor-triggered retries, and Cost per run reports the total commercial-API spend for the full evaluation set, fixed at zero for open-source and fine-tuned configurations.

Table 6.3: Custom operational metrics for MultiWOZ.

All custom metrics are computed on the lexicalized form of the system response. Booking Rate is averaged over dialogues; all other metrics are micro-averaged over user turns, excluding turns where the metric does not apply.

Metric	**Level**	**What it measures**
Domain Precision	turn	fraction of user turns whose predicted active domain matches the ground-truth domain
Intent Precision	turn	fraction of user turns whose predicted active intent matches the ground-truth intent
Action Accuracy	turn	fraction of user turns where the predicted next system action (*inform* / *request* / *book*, derived from the predicted intent and policy violations) matches the ground-truth action inferred from the following system turn's dialog-act labels
Joint Goal Accuracy (JGA)	turn	fraction of user turns whose full predicted accumulated belief state matches the ground truth exactly
Slot Recall	turn	fraction of ground-truth slot–value pairs correctly recovered by the predicted belief state
Slot F1	turn	harmonic mean of slot precision and slot recall against the ground-truth belief state
Hallucination Rate	turn	fraction of turns whose lexicalized response mentions an entity not returned by the database lookup of that turn (averaged only over turns that mention an entity)
Policy Violation Rate	turn	fraction of turns where a booking intent is acted upon despite at least one required booking slot being missing
System Correctness	turn	fraction of turns with neither a hallucination nor a policy violation
Booking Rate	dialogue	fraction of dialogues with at least one booking intent in which a booking confirmation is reached (dialogues without booking intent are excluded from the average)
Latency per turn	turn	average wall-clock time per user turn in seconds, including supervisor-triggered retries
Cost per run	turn	total commercial-API spend for the full evaluation set in USD (zero for open-source and fine-tuned configurations)

The custom metrics are not optional. The standardized metrics of Inform, Success, BLEU, Combined were designed for end-to-end evaluation against a fixed reference and do not, on their own, distinguish between the kinds of failure that the architectures of this work are most likely to produce: a hallucinated entity name and a missing requested slot can both lower Success Rate without revealing which of the two occurred, and neither of them tells the reader whether the failure originated in DST or in response generation. The custom metrics are designed precisely to surface these distinctions, and the recommendation of Deriu et al. [22]

that dialogue-system evaluation should combine a small set of widely accepted standard metrics with task-specific operational measures, is what motivates reporting both families side by side throughout the next chapter.

7. Results and Discussion

This chapter reports the numerical results of the evaluation defined in Chapter 6. It walks through Experiment 1, Experiment 2, and Experiment 3 in turn, each with its aggregate metrics, and a discussion of the patterns those numbers expose. It then synthesizes the architectural trajectory across the three Experiments, positions the resulting numbers against published baselines on MultiWOZ where comparison is feasible, examines the residual failure modes through a turn-level error analysis, and closes with the methodological limitations that bound the conclusions. Both metric families from Chapter 6 are reported throughout as the work treats both as necessary.

7.1 Experiment 1: Single-LLM Baseline

Experiment 1 places a single general-purpose LLM into the monolithic role described in Chapter 5. Of the configurations attempted at the monolithic role, nine produced complete result sets and are reported in the tables below.

7.1.1 Excluded Configurations

Two configurations attempted at the monolithic role of Experiment 1 did not produce evaluatable output. Both are parse-time failures: the model returns a string in response to the monolithic prompt, but the four-field JSON object specified in Chapter 5 cannot be recovered from it, and the turn collapses into a generic fallback with no domain, no intent, no slots, and no response. Mistral-Nemo-12B fails by being too verbose. It produces a syntactically valid four-field JSON object on every turn, but keeps writing after the closing brace, adding conversational text of the form {...} *Here is another option you might consider: ....* The strict parser reads the JSON object correctly, then rejects the trailing text as extra data. This happens on every turn because Mistral-Nemo was trained to add explanations and does so even when told not to. LLaMA-3.2-3B-Instruct fails the opposite way. At 3.2 billion parameters it is too small to follow a strict JSON-only instruction in a single zero-shot prompt, and falls back to the plain-text key-value format that dominates its pretraining, typically emitting a single value such as *null* followed by plain-text headers like *DOMAIN: hotel\nINTENT: find_hotel*. The parser reads the first token as a valid JSON value, then rejects everything after as extra data. The failure happens within the first few characters of every response since the model abandons the JSON format almost immediately. Both failure modes are characteristic of zero-shot prompting for strictly structured output, a verbose model producing too much around the object and a small model producing too little of it, and they motivated the choice of the Qwen family as the lead Experiment 1 backbones, since Qwen2.5 [7] and Qwen3 [8] are trained with explicit structured-output objectives and produce clean four-field JSON objects without prefix or continuation under the same prompt.

The two configurations are excluded from the tables that follow. LLaMA-3.2-3B is reported again later in the chapter. In Experiment 2, the modular pipeline replaces the monolithic four-field prompt with shorter per-role prompts that the model can follow. In Experiment 3, LoRA fine-tuning supplies the structured-output discipline that the model could not produce zero-shot.

7.1.2 Overall Results

Tables 7.1 and 7.2 report the metrics produced by the nine running configurations of Experiment 1 on the 186 dialogues and 1,100 user turns of the MultiWOZ 2.2 test split filtered to the hotel and restaurant domains. Table 7.1 reports the operational metrics, namely the per-turn classification, reliability, and bookkeeping diagnostics of Chapter 6, together with total cost and per-turn latency. Table 7.2 reports the six metrics that the MultiWOZ literature uses for cross-system comparison: JGA and Slot F1 from the custom family, joined with Inform Rate, Success Rate, BLEU, and Combined from the standardized family. The column set of Table 7.2 is the same one used in the literature-comparison tables later in the chapter. Both tables apply the per-turn exclusions defined in Chapter 6.

Table 7.1: Custom operational metrics of Experiment 1.

All metrics in %, except Cost in US dollars and Latency in seconds. Same convention applies to all results tables in this chapter.

Config	DomP	IntP	Act	SlotR	Hall	Pol	SysCorr	Book	Cost	Latency
gpt	98.7	87.0	77.6	64.7	5.9	0.8	94.9	57.4	2.75	3.25
gpt-nano	96.9	84.1	75.3	74.6	12.4	0.3	97.2	33.8	1.80	2.10
haiku	98.8	90.1	82.8	84.3	4.4	3.6	92.6	76.7	21.03	3.04
qw3_4b	98.0	83.9	65.9	51.4	13.2	16.6	75.2	34.7	0	7.82
qw3_8b	99.2	88.5	74.1	78.6	9.7	2.2	94.5	64.3	0	7.72
qw25_14b	98.1	92.9	77.7	68.7	5.7	0.7	94.3	57.9	0	13.28
qw3_14b	98.7	89.3	78.4	78.6	2.7	1.9	96.0	75.8	0	12.40
lla31_8b	90.8	67.9	67.8	37.0	53.7	0.1	87.0	0	0	7.22
phi4_14b	97.0	83.7	78.8	74.0	7.2	5.1	90.5	74.8	0	13.81

Table 7.2: Standard MultiWOZ leaderboard metrics of Experiment 1.

Config	JGA	SlotF1	Inform	Success	BLEU	Combined
gpt	24.8	71.2	58.6	44.1	3.08	54.43
gpt-nano	31.4	78.7	33.9	19.4	3.96	30.61
haiku	44.8	88.3	90.9	83.3	3.67	90.77
qw3_4b	14.8	54.7	50.5	39.8	2.59	47.74
qw3_8b	32.3	82.1	43.0	31.2	4.48	41.58
qw25_14b	26.4	74.9	74.2	54.8	3.37	67.87
qw3_14b	33.3	83.0	82.3	72.0	3.69	80.84
lla31_8b	5.2	33.1	14.0	2.2	2.30	10.40
phi4_14b	25.8	74.3	66.1	59.7	4.46	67.36

7.1.3 Discussion

In Table 7.2, Claude Haiku 4.5 reaches the highest Combined score at 90.77, followed by Qwen3-14B at 80.84; LLaMA-3.1-8B finishes lowest at 10.40. Claude Haiku 4.5 also leads on JGA at 44.8 and Slot F1 at 88.3, with Qwen3-14B second at 33.3 and 83.0. Under a single architecture, Combined varies from 90.77 down to 10.40 across the nine models. In Experiment 1, model choice is therefore the dominant factor in end-to-end quality.

The three commercial APIs cover Combined scores of 90.77 (Claude Haiku 4.5), 54.43 (GPT-4o-mini), and 30.61 (GPT-4.1-nano), with costs of $21.03, $2.75, and $1.80 respectively. Within the two cheaper APIs, GPT-4.1-nano tracks the user better than GPT-4o-mini does (JGA 31.4 against 24.8, Slot F1 78.7 against 71.2) yet acts on what it tracks worse (Inform 33.9 against 58.6, Success 19.4 against 44.1). In the monolithic single-LLM role, knowing what the user wants and delivering on it are therefore different capabilities, and the cheaper API can be good at one without the other.

In the 14B class, family separates the three configurations. Qwen3-14B reaches Combined 80.84, while Qwen2.5-14B finishes at 67.87 and Phi-4-14B at 67.36. Below 14B, the smaller Qwen3-4B at Combined 47.74 outscores the larger LLaMA-3.1-8B at 10.40. Within the Qwen3 family, Combined does not rise with parameter count. Qwen3-4B sits above Qwen3-8B at 41.58, while Qwen3-14B at 80.84 sits above both. The slot metrics within Qwen3 do follow size, with Qwen3-8B at JGA 32.3 and Slot F1 82.1 against Qwen3-4B at JGA 14.8 and Slot F1 54.7, while Qwen3-14B sits again on top for both JGA and Slot F1. On Inform and Success, however, Qwen3-8B posts 43.0 and 31.2 against Qwen3-4B's 50.5 and 39.8, which is what pulls Qwen3-8B's Combined below Qwen3-4B's. In the single-LLM role of Experiment 1, model family separates the configurations more than parameter count does.

Two configurations need individual treatment for different reasons. LLaMA-3.1-8B is lowest on every metric in Tables 7.1 and 7.2 (JGA 5.2, Slot F1 33.1, Inform 14.0, Success 2.2, Combined 10.40), with a hallucination rate of 53.7%, the highest in the Experiment. Its scores are too far below the others to support meaningful Combined comparisons. Phi-4-14B is competitive on task quality (Combined 67.36, JGA 25.8, Slot F1 74.3) but is the slowest open-source configuration at 13.81 seconds per turn, ahead of Qwen3-14B at 12.40 seconds despite a similar parameter count. The monolithic prompt of Experiment 1 exceeds the 16K native context window of Phi-4-14B, and the inference stack applies a 2× RoPE scaling factor to extend the position embeddings to 32K. This raises the per-token cost above native-length inference. LLaMA-3.1-8B is the outlier of the quality columns. Phi-4-14B is the outlier of the latency column.

Hallucination rate in Table 7.1 has LLaMA-3.1-8B at 53.7%, far above every other configuration. With LLaMA-3.1-8B set aside, the eight remaining configurations all hallucinate below 14% and six of them below 10%. The lowest are Qwen3-14B at 2.7% and Claude Haiku 4.5 at 4.4%, and the highest are Qwen3-4B at 13.2% and GPT-4.1-nano at 12.4%. The order on this column tracks Combined. The four configurations with Combined above 67 (Claude Haiku 4.5, Qwen3-14B, Qwen2.5-14B, Phi-4-14B) all hallucinate below 8%, while the three configurations with Combined below 50 (Qwen3-8B, Qwen3-4B, GPT-4.1-nano) all hallucinate at 9.7% or above. GPT-4o-mini is the exception at Combined 54.43 and hallucination 5.9%, with Combined between the two groups but hallucination below 8% along with the high-Combined group. In the single-LLM role of Experiment 1, hallucination correlates with Combined but carries additional information about the configuration that Combined alone does not.

Cost in Table 7.1 separates by access type. The six open-source configurations all cost zero on the 1,100-turn evaluation. The three commercial APIs cost $21.03 (Claude Haiku 4.5), $2.75 (GPT-4o-mini), and $1.80 (GPT-4.1-nano). Latency separates by both access type and parameter count. The three APIs run fastest (GPT-4.1-nano 2.10 seconds per turn, Claude Haiku 4.5 3.04, GPT-4o-mini 3.25). The 4B and 8B open-source configurations follow at 7.22 to

7.82 seconds. The three 14B-class open-source configurations are slowest at 12.40 to 13.81 seconds. Among the strongest configurations of the Experiment, the API option (Claude Haiku 4.5) is the fastest and highest-quality but costs the most, while the open-source option (Qwen3-14B) is zero-cost but slower and lower-quality. The three-way trade-off between cost, latency, and quality defines the single-LLM role of Experiment 1.

The nine configurations of Experiment 1 motivate the modular pipeline tested in Experiment 2 along four numbered patterns. First, the strongest open-source configuration is Qwen3-14B at Combined 80.84, while the strongest API is Claude Haiku 4.5 at 90.77. Experiment 2 tests whether per-role prompts let a strong open-source model close the gap to Claude Haiku 4.5. Second, GPT-4.1-nano and Qwen3-8B track the user's slots better than they fulfill the user's request (GPT-4.1-nano JGA 31.4 against Inform 33.9, Qwen3-8B JGA 32.3 against Inform 43.0). Experiment 2 separates slot tracking from response generation and tests whether the same configurations still track better than they deliver. Third, the three open-source configurations below 14B all finish below Combined 50 (Qwen3-4B 47.74, Qwen3-8B 41.58, LLaMA-3.1-8B 10.40). Experiment 2 reduces the per-role prompt to a length each model can handle and tests whether smaller models become viable. Fourth, Claude Haiku 4.5 reaches $21.03 in API cost and the 14B-class open-source configurations reach 12.40 to 13.81 seconds per turn in latency. Experiment 2 tests whether the shorter per-role prompts reduce cost and latency for these configurations.

7.2 Experiment 2: Zero-Shot Modular Pipeline

Experiment 2 replaces the monolithic role of Experiment 1 with the modular pipeline described in Chapter 5, with both LLM modules used zero-shot. The only architectural difference from Experiment 1 is the decomposition into per-role prompts, so the numbers in the next subsection isolate what that decomposition produces by itself. Thirteen configurations were evaluated, four using commercial APIs and nine using open-source backbones, and all of them produced complete result sets.

7.2.1 Overall Results

Tables 7.3 and 7.4 report the metrics produced by the thirteen configurations of Experiment 2 on the same test split as Experiment 1. The column structure mirrors Tables 7.1 and 7.2: Table 7.3 reports the operational diagnostics together with cost and latency, and Table 7.4 reports the six standard MultiWOZ leaderboard metrics. Configuration names follow the convention homo_X for a configuration where the same backbone X fills both the DST and ResponseGen roles, and het_X_Y for a configuration where backbone X fills the DST role and backbone Y fills the ResponseGen role. The model labels match Table 6.1.

Table 7.3: Custom operational metrics of Experiment 2.

Config	DomP	IntP	Act	SlotR	Hall	Pol	SysCorr	Book	Cost	Latency
homo_gpt	99.6	90.9	80.2	80.1	3.1	3.4	94.6	65.9	0.20	4.00
homo_haiku	99.8	91.0	81.1	84.0	0.8	5.1	94.4	75.2	2.10	4.73
het_gpt_haiku	99.8	91.5	80.2	80.3	1.0	4.7	94.5	69.7	0.78	3.70

het_haiku_gpt	99.8	91.2	80.7	83.8	2.1	3.7	95.4	71.4	1.53	4.86
homo_lla32_3b	86.4	69.9	65.5	40.5	14.2	4.9	89.7	22.1	0	3.17
homo_qw3_4b	99.3	85.4	73.1	76.9	3.2	5.2	93.0	63.3	0	2.26
homo_qw3_8b	99.4	91.4	78.2	79.0	11.2	2.8	94.5	61.8	0	2.18
homo_lla31_8b	96.9	85.8	76.0	60.3	5.1	1.8	95.2	40.7	0	3.26
homo_qw3_14b	99.8	89.1	77.9	79.9	3.0	1.1	97.1	70.0	0	2.50
homo_phi4_14b	99.3	91.8	76.8	73.8	9.2	9.1	88.9	46.1	0	2.75
het_qw25_qw3_14b	99.4	88.7	76.4	77.2	4.2	2.4	95.4	58.5	0	2.56
het_qw3_14b_qw3_8b	99.6	91.1	81.6	80.2	10.9	2.2	94.8	68.0	0	2.46
het_qw3_14b_phi4_14b	99.6	91.6	80.4	80.8	7.9	3.0	93.9	72.8	0	2.88

Table 7.4: Standard MultiWOZ leaderboard metrics of Experiment 2.

Config	JGA	SlotF1	Inform	Success	BLEU	Combined
homo_gpt	35.7	81.7	79.0	72.0	3.14	78.64
homo_haiku	45.1	87.3	90.3	83.9	4.25	91.35
het_gpt_haiku	34.7	81.9	78.0	72.6	3.92	79.22
het_haiku_gpt	44.6	87.0	88.2	81.2	2.95	87.65
homo_lla32_3b	5.3	42.5	28.5	14.5	1.26	22.76
homo_qw3_4b	26.7	77.2	68.3	59.7	3.11	67.11
homo_qw3_8b	34.2	81.3	51.6	42.5	5.10	52.15
homo_lla31_8b	18.1	63.4	55.4	35.5	1.53	46.98
homo_qw3_14b	33.4	80.4	73.7	68.3	5.00	76.00
homo_phi4_14b	23.8	69.7	61.3	43.5	2.83	55.23
het_qw25_qw3_14b	30.1	76.9	73.7	60.8	4.54	71.79
het_qw3_14b_qw3_8b	33.7	81.6	52.7	45.2	5.20	54.15
het_qw3_14b_phi4_14b	33.7	81.1	68.3	62.9	2.91	68.51

7.2.2 Discussion

In Table 7.4, homo_haiku reaches the highest Combined score at 91.35, followed by het_haiku_gpt at 87.65 and homo_gpt at 78.64. The strongest fully open-source configuration is homo_qw3_14b at Combined 76.00. homo_lla32_3b finishes lowest at 22.76. homo_haiku also leads on JGA at 45.1 and Slot F1 at 87.3, with het_haiku_gpt second at 44.6 and 87.0. Under the modular pipeline, the four configurations using GPT-4o-mini or Claude Haiku 4.5 in at least one role occupy the top four positions on Combined, with homo_qw3_14b leading the fully open-source group below them.

Comparing each homogeneous Experiment 2 configuration's Combined score against its Experiment 1 counterpart in Table 7.2 splits the inventory into two groups. The weaker Experiment 1 configurations gain from the modular pipeline. LLaMA-3.1-8B rises from 10.40 to 46.98, GPT-4o-mini from 54.43 to 78.64, Qwen3-4B from 47.74 to 67.11, and Qwen3-8B from 41.58 to 52.15. Claude Haiku 4.5, already near the top of Experiment 1 at 90.77, rises only slightly to 91.35. The stronger Experiment 1 configurations lose ground. Qwen3-14B falls from 80.84 to 76.00 and Phi-4-14B falls from 67.36 to 55.23. The predictor of which group a model falls into is its Experiment 1 Combined score. Weak models gain because the shorter per-role prompts of Experiment 2 fit each sub-task into capacity. Strong models lose because the per-role split removes the cross-role context they were using productively in the single pass of Experiment 1.

The heterogeneous configurations test which role drives Combined by taking a homogeneous configuration as the starting point, replacing one role at a time with a stronger model, and checking whether Combined improves. Starting from homo_gpt at Combined 78.64, replacing the dialogue state tracker alone with Claude Haiku 4.5 lifts the configuration to het_haiku_gpt at 87.65, while replacing the response generator alone with Claude Haiku 4.5 lifts it only to het_gpt_haiku at 79.22. The DST upgrade therefore improves Combined far more than the RG upgrade, identifying dialogue state tracking as the bottleneck of homo_gpt. The cross-family pair confirms the asymmetry. Starting from homo_phi4_14b at Combined 55.23, replacing Phi-4-14B's dialogue state tracker with Qwen3-14B's while keeping Phi-4-14B for response generation lifts the configuration to het_qw3_14b_phi4_14b at 68.51, closing most of the gap to homo_qw3_14b at 76.00. In the modular pipeline of Experiment 2, dialogue state tracking is therefore the dominant role of the pipeline, and a strong dialogue state tracker recovers most of the Combined that a weak DST gives away.

Hallucination rate in Table 7.3 redistributes against Table 7.1. The high-hallucination Experiment 1 configurations drop sharply. LLaMA-3.1-8B moves from 53.7% to 5.1%, Qwen3-4B from 13.2% to 3.2%, GPT-4o-mini from 5.9% to 3.1%, and Claude Haiku 4.5 from 4.4% to 0.8%. The already-disciplined Experiment 1 configurations stay flat or rise. Qwen3-14B goes from 2.7% to 3.0%, Phi-4-14B from 7.2% to 9.2%, and Qwen3-8B from 9.7% to 11.2%. Within Experiment 2, hallucination groups by the response generator model. Configurations using Claude Haiku 4.5 as response generator are at 0.8% and 1.0%, GPT-4o-mini at 2.1% and 3.1%, Qwen3-14B at 3.0% and 4.2%, Phi-4-14B at 7.9% and 9.2%, and Qwen3-8B at 10.9% and 11.2%. homo_lla32_3b sits highest at 14.2%. With Qwen3-14B held constant as the dialogue state tracker, hallucination is 3.0% when the response generator is Qwen3-14B (homo_qw3_14b) but rises to 10.9% when the response generator is Qwen3-8B (het_qw3_14b_qw3_8b). Changing the dialogue state tracker while holding the response generator constant moves the rate by much less. In the modular pipeline of Experiment 2, the response generator is therefore the primary determinant of hallucination, with the dialogue state tracker as a smaller secondary contributor.

Cost in Table 7.3 drops sharply against Table 7.1 for the API configurations. homo_gpt costs $0.20 against GPT-4o-mini's $2.75 in Experiment 1, and homo_haiku costs $2.10 against Claude Haiku 4.5's $21.03. Each per-role prompt of Experiment 2 is shorter than the monolithic prompt of Experiment 1, and total tokens billed fall accordingly. Latency moves in opposite directions for API and open-source configurations. The API homogeneous configurations slow down. homo_gpt rises from 3.25 seconds per turn in Experiment 1 to 4.00, and homo_haiku from 3.04 to 4.73. The modular pipeline issues two network calls per turn against one in Experiment 1, and most of each call's time goes to the network, not to the model's work. The open-source configurations speed up. homo_qw3_14b drops from 12.40 to 2.50, homo_phi4_14b from 13.81 to 2.75, and homo_qw3_4b from 7.82 to 2.26. The per-role prompts are short enough that the per-call compute savings more than offset the doubling of calls. Prompt length therefore drives both cost and latency in Experiment 2, while access type determines whether latency moves up or down.

The thirteen configurations of Experiment 2 motivate the LoRA fine-tuning tested in Experiment 3 along three numbered patterns. First, the strongest open-source configuration is homo_qw3_14b at Combined 76.00, while the strongest API is homo_haiku at 91.35. The architectural change from Experiment 1 to Experiment 2 helped weak models but hurt the strongest open-source ones, which means that further closing the API gap from open-source models has to come from the models themselves rather than from architecture. Experiment 3 tests whether parameter-efficient fine-tuning can supply that. Second, the role-asymmetry analysis in this Experiment identified dialogue state tracking as the dominant role of the modular pipeline. Experiment 3 fits a role-specific LoRA adapter to dialogue state tracking, where the supervised target is a clean structured output. Third, the configurations using Qwen3-8B, Phi-4-14B, or LLaMA-3.2-3B as the response generator hallucinate at 7.9% or above. Experiment 3 fits a role-specific LoRA adapter to response generation as well, on the hypothesis that supervised training can lower hallucination.

7.3 Experiment 3: LoRA Fine-Tunes Modular Pipeline

Experiment 3 keeps the modular pipeline of Experiment 2 unchanged. The difference is that each LLM module is now a role-specific LoRA adapter fine-tuned on the MultiWOZ training split via QLoRA, so the only difference from Experiment 2 is role-specific specialization. Seven configurations were evaluated, all open-source: six homogeneous and one heterogeneous pairing Qwen3-14B's DST adapter with Phi-4-14B's response-generator adapter; commercial APIs are absent because their weights are not exposed for fine-tuning.

7.3.1 Overall Results

Tables 7.5 and 7.6 report the metrics produced by the seven configurations of Experiment 3 on the same test split as Experiments 1 and 2. The column structure mirrors Tables 7.3 and 7.4: Table 7.5 reports the operational diagnostics together with cost and latency, and Table 7.6 reports the six standard MultiWOZ leaderboard metrics. Configuration names extend the Experiment 2 convention with the ft_ prefix to indicate fine-tuned: ft_homo_X for a configuration where backbone X carries two role-specific adapters (one DST adapter and one ResponseGen adapter, both trained on the same base model), and ft_het_X_Y for a configuration where backbone X carries the DST adapter and backbone Y carries the ResponseGen adapter.

Table 7.5: Custom operational metrics of Experiment 3.

Config	DomP	IntP	Act	SlotR	Hall	Pol	SysCorr	Book	Cost	Latency
ft_homo_lla32_3b	98.2	94.3	77.1	87.5	27.9	0.3	90.6	44.6	0	2.12
ft_homo_qw3_4b	99.7	95.2	78.3	86.2	16.2	0.7	94.3	43.0	0	2.78
ft_homo_qw3_8b	99.3	95.7	78.9	88.1	22.1	0.6	92.6	48.3	0	3.36
ft_homo_lla31_8b	96.4	92.5	75.8	86.1	45.6	1.0	84.2	50.8	0	3.14
ft_homo_qw3_14b	99.0	95.7	79.2	88.1	14.1	2.0	94.7	54.2	0	3.42
ft_homo_phi4_14b	99.1	94.5	78.6	88.7	15.5	0.8	94.9	55.0	0	3.44
ft_het_qw3_phi4_14b	99.0	95.6	78.7	87.7	13.8	1.2	94.9	55.7	0	3.51

Table 7.6: Standard MultiWOZ leaderboard metrics of Experiment 3.

Config	JGA	SlotF1	Inform	Success	BLEU	Combined
ft_homo_lla32_3b	43.5	87.8	57.0	39.8	9.39	57.79
ft_homo_qw3_4b	44.8	88.3	54.8	36.0	8.35	53.75
ft_homo_qw3_8b	47.3	89.1	53.2	48.4	11.35	62.15
ft_homo_lla31_8b	44.0	87.1	50.5	40.3	9.22	54.62
ft_homo_qw3_14b	47.7	89.5	61.3	55.4	12.26	70.61

ft_homo_phi4_14b	47.0	88.9	66.7	57.5	8.74	70.84
ft_het_qw3_phi4_14b	46.9	89.2	66.7	57.0	8.71	70.56

7.3.2 Discussion

In Table 7.6, the three highest-scoring configurations of Experiment 3 cluster within 0.3 points: ft_homo_phi4_14b at 70.84, ft_homo_qw3_14b at 70.61, and ft_het_qw3_phi4_14b at 70.56, all three below homo_haiku of Experiment 2 at 91.35. The lowest-scoring of Experiment 3 is ft_homo_qw3_4b at 53.75. The DST column of Table 7.6 ranks differently. ft_homo_qw3_14b leads on both JGA at 47.7 and Slot F1 at 89.5, ahead of ft_homo_phi4_14b at JGA 47.0 and Slot F1 88.9. JGA across the seven configurations runs from 43.5 to 47.7, while Slot F1 runs from 87.1 to 89.5. All seven configurations run at zero dollars, with per-turn latency between 2.12 and 3.51 seconds. The Combined column varies more across the seven configurations than the JGA column does, and the rank order on Combined within Experiment 3 is therefore set by the response-generation half of the pipeline rather than by DST.

Comparing each homogeneous Experiment 3 configuration's Combined score against its Experiment 2 counterpart in Table 7.4 splits the inventory by Experiment 2 hallucination rate rather than by parameter size. The configurations that hallucinated above 9% in Experiment 2 gain from fine-tuning. LLaMA-3.2-3B rises from 22.76 to 57.79, Phi-4-14B from 55.23 to 70.84, and Qwen3-8B from 52.15 to 62.15. The configurations that hallucinated below 4% in Experiment 2 lose ground. Qwen3-4B falls from 67.11 to 53.75 and Qwen3-14B falls from 76.00 to 70.61. LLaMA-3.1-8B sits between the two groups with an Experiment 2 hallucination rate of 5.1%, and rises modestly from 46.98 to 54.62. The contrast between Phi-4-14B and Qwen3-4B confirms that the predictor is not model size. The larger Phi-4-14B had high Experiment 2 hallucination and gains the most among 14B configurations, while the smaller Qwen3-4B was already disciplined and loses the most. Fine-tuning helps when the base needed corrective signal it was not receiving from prompting alone, and hurts when the base was already producing the right outputs zero-shot.

The DST half of the pipeline closes the gap to the commercial APIs, but the response generator half does not. ft_homo_qw3_14b reaches JGA 47.7 and Slot F1 89.5, both above homo_haiku of Experiment 2 at JGA 45.1 and Slot F1 87.3. The fine-tuned 14B open-source DST therefore exceeds Claude Haiku 4.5 zero-shot on both DST headline metrics, at zero dollars and 3.42 seconds per turn. The Combined gap remains, however, ft_homo_qw3_14b finishes at Combined 70.61 against homo_haiku at 91.35, and the difference is concentrated in response generation. Inform falls from Claude Haiku 4.5's 90.3 to ft_homo_qw3_14b's 61.3, and Success from 83.9 to 55.4. BLEU rises from 4.25 to 12.26, because the fine-tuned response generator learns to mimic the surface forms of the training references. ft_homo_phi4_14b is the partial exception at Combined 70.84, the highest in Experiment 3, with Inform 66.7 and Success 57.5. After fine-tuning, the bottleneck of the pipeline has moved from dialogue state tracking to response generation, with Phi-4-14B as the only base that improves both halves at once.

The single heterogeneous configuration of Experiment 3 splits cleanly along role lines. ft_het_qw3_phi4_14b combines Qwen3-14B as the dialogue state tracker with Phi-4-14B as the response generator. Its dialogue state tracking metrics match the Qwen3-14B homogeneous configuration. ft_het_qw3_phi4_14b reaches JGA 46.9 and Slot F1 89.2, against ft_homo_qw3_14b at 47.7 and 89.5. Its end-to-end metrics match the Phi-4-14B homogeneous configuration. ft_het_qw3_phi4_14b reaches Inform 66.7 and Success 57.0, against ft_homo_phi4_14b at 66.7 and 57.5. The hetero finishes at Combined 70.56, within 0.3 points of both homogeneous components (ft_homo_qw3_14b at 70.61 and ft_homo_phi4_14b at 70.84). After fine-tuning, each role-specific adapter therefore controls its half of the pipeline almost independently, and a heterogeneous configuration takes its slot metrics from one role-adapter and its end-to-end metrics from the other.

Hallucination rate rises across all seven configurations of Experiment 3 against their Experiment 2 counterparts. LLaMA-3.1-8B moves from 5.1% to 45.6%, LLaMA-3.2-3B from 14.2% to 27.9%, Qwen3-8B from 11.2% to 22.1%, Qwen3-4B from 3.2% to 16.2%, Qwen3-14B from 3.0% to 14.1%, and Phi-4-14B from 9.2% to 15.5%. The LLaMA family rises the most. ft_homo_lla31_8b at 45.6% and ft_homo_lla32_3b at 27.9% are the two highest hallucinators of Experiment 3. The three configurations using only the 14B models Qwen3-14B and Phi-4-14B cluster at the bottom of the column. ft_het_qw3_phi4_14b is at 13.8%, ft_homo_qw3_14b at 14.1%, and ft_homo_phi4_14b at 15.5%. For context, the highest hallucination rate in Experiment 2 was 14.2% (homo_lla32_3b). Six of the seven Experiment 3 configurations hallucinate above this rate. After fine-tuning, the seven configurations therefore sit well above the Experiment 2 baseline on this column, even at its best.

Cost in Table 7.5 is zero across all seven configurations of Experiment 3. Per-turn latency runs from 2.12 (ft_homo_lla32_3b) to 3.51 seconds (ft_het_qw3_phi4_14b). The latency range matches the Experiment 1 commercial-API range of 2.10 to 3.25 seconds, while costing nothing against the $1.80 to $21.03 of those APIs. The latency range also matches the Experiment 2 open-source range of 2.18 to 3.26 seconds, since the LoRA adapters do not change the inference path of the modular pipeline. After fine-tuning, the open-source modular pipeline therefore matches commercial API economics on cost and latency, while keeping the per-call compute on local hardware.

The seven configurations of Experiment 3 deliver three findings about role-specific LoRA fine-tuning. First, the dialogue state tracking adapter brings open-source models to API-comparable DST quality at zero cost. ft_homo_qw3_14b reaches JGA 47.7 and Slot F1 89.5, both above homo_haiku of Experiment 2 and Claude Haiku 4.5 of Experiment 1. Second, the response generator adapter does not bring the open-source pipeline to API-comparable end-to-end quality. The best Experiment 3 Combined of 70.84 (ft_homo_phi4_14b) sits well below homo_haiku's 91.35, with the gap concentrated in Inform and Success. Third, hallucination rises across every configuration. ft_homo_lla31_8b at 45.6% is the most affected, while even the lowest, ft_het_qw3_phi4_14b at 13.8%, is near the highest hallucination rate in Experiment 2 (14.2%).

7.4 Cross-Experiment Synthesis

Sections 7.1 through 7.3 read each Experiment on its own. This section follows each model's full path from Experiment 1 to Experiment 2 to Experiment 3, treating the three points as a single trajectory. Five open-source models ran in all three Experiments and are tracked here: Qwen3-4B, Qwen3-8B, Qwen3-14B, LLaMA-3.1-8B, and Phi-4-14B. The three commercial APIs are excluded because their weights are not exposed for fine-tuning. LLaMA-3.2-3B is excluded because its Experiment 1 output could not be parsed, and Qwen2.5-14B because it was not enrolled in Experiment 3.

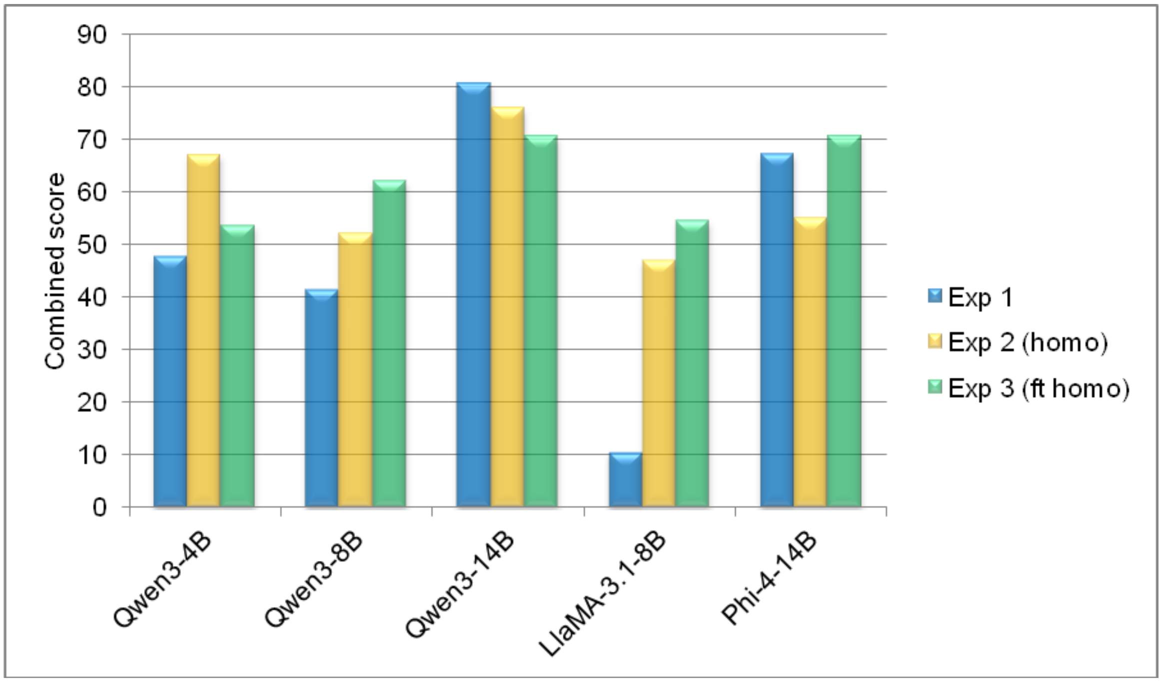

Figure 7.1: Combined score per model across the three Experiments.

Figure 7.1 shows four trajectory shapes across the five models. Two models rise across the three Experiments: LLaMA-3.1-8B moves from Combined 10.40 to 46.98 to 54.62, and Qwen3-8B moves from 41.58 to 52.15 to 62.15. Both were too weak in Experiment 1 for a single prompt to handle the entire turn, and both had Experiment 2 hallucination, LLaMA-3.1-8B at 5.1% and Qwen3-8B at 11.2%, that fine-tuning could correct. Qwen3-4B peaks in Experiment 2 at 67.11, with 47.74 in Experiment 1 and 53.75 in Experiment 3. The per-role prompts of Experiment 2 fit its capacity, and its Experiment 2 hallucination of 3.2% was low enough that fine-tuning had nothing to correct. Qwen3-14B falls across the three Experiments, from 80.84 to 76.00 to 70.61, the only case where the strongest configuration of the model is its monolithic one. It was strong enough in Experiment 1 to lose from decomposition, and its Experiment 2 hallucination of 3.0% gave fine-tuning nothing to correct either. Phi-4-14B traces a V, from 67.36 down to 55.23 and back up to 70.84, the only model whose Experiment 3 score exceeds its Experiment 1 score. It shares the first half of Qwen3-14B's pattern, strong in Experiment 1 and losing from decomposition, but its Experiment 2 hallucination of 9.2% gave fine-tuning errors to correct.

The trajectories of the five models reveal two architectural levers with different primary effects. Decomposition is the cost-and-latency lever: per-role prompts are shorter than the monolithic prompt, so API spend and open-source per-turn latency both fall sharply in Experiment 2, including for models whose quality fell. Specialization is the DST lever: fine-tuned 14B-class adapters exceed the Claude Haiku 4.5 zero-shot baseline on Joint Goal Accuracy and Slot F1 (Section 7.3.2), but specialization does not lift response generation to the same level. No configuration reaches Claude Haiku 4.5's 91.35 at any point on any trajectory. The closest single point is Qwen3-14B in Experiment 1 at 80.84, and the closest fine-tuned point is Phi-4-14B in Experiment 3 at 70.84. The gap therefore sits in response generation, not in dialogue state tracking.

7.5 Error Analysis

Across the three experiments the same kinds of mistakes keep appearing in different configurations and at different rates. Each one comes from a specific place in the pipeline where a design tradeoff or a model behavior breaks down. This section walks through the recurring error types observed on the test set. Each subsection defines one error in simple words, says where it shows up across the experiments, gives the architectural reason, and quotes one short turn from the test set as an illustration.

A multi-domain turn confusion happens when the user mentions two domains in one sentence but the pipeline picks only one of them and drops the slots of the other. It appears in every configuration of all three Experiments at a similar rate because it is a design choice rather than a model weakness, and it affects roughly three to four percent of turns on the MultiWOZ 2.2 test split. The cause is architectural. The pipeline is built to pick one domain per turn, so when a user combines a hotel and a restaurant request in the same utterance only one set of slots is added to the predicted state for that turn. The missing slots from the other domain cause the Joint Goal Accuracy to fail for that turn and for every later turn, until the missing slot resurfaces in a later user utterance. In dialogue PMUL0079 the user says "Okay. now could you help me find a restaurant in the expensive price range that is in the same area as the hotel?", the ground truth marks both hotel and restaurant as active, but the pipeline predicts restaurant only, so the hotel state is never refreshed for that turn.

Slot extraction and cumulative-state errors are cases where the dialogue state tracker records a slot value that does not match the ground truth. Because the belief state accumulates across all turns of a dialogue, one wrong extraction causes the Joint Goal Accuracy to fail for that turn and every later one. This is the dominant source of JGA failure in every configuration of all three Experiments. JGA stays well below half on every model, and even the best fine-tuned configuration in Experiment 3, ft_homo_qw3_14b, only reaches 47.7 percent. The mistakes come in three common shapes. The model picks the wrong slot key, or it normalizes the value differently from the dataset (for example "5 nights" instead of "5"), or it misses a slot the user actually provided. The cumulative penalty turns each one of these small mistakes into a permanent JGA failure for the rest of the dialogue. In dialogue SNG0613 the user says "I would like to try saigon city." The ground truth records this as restaurant-name equal to "saigon city" because saigon city is a restaurant in the database. The pipeline records it as restaurant-area equal to "saigon city" instead, because the model sees the word city and treats it as a location. From that turn onwards the belief state is wrong and the dialogue cannot recover.

A hallucination is a response that mentions an entity, address, or phone number that does not come from the database results passed to the response generator. Three architectural subtypes appear across the experiments. DB-vs-prompt mismatch only shows up in Experiment 1, because the single LLM sees the database and the user query in one prompt and sometimes ignores the database, generating details from its training data instead. Empty-DB mention appears in all three Experiments and happens when the database lookup returned no entity but the response still names one. Booking confirmation also appears in all three Experiments and happens when the response confirms a reservation and emits a [ref] token even though the entity is not in the database results passed to the response generator. In dialogue PMUL4643 the user says "Please book a table for 5 at 14:30 on Wednesday at Royal Spice. I will need the reference number. I also need to find a place to stay." Because the user mentions both a restaurant and a hotel in one turn, the pipeline picks only the hotel side, queries the hotel database, and passes a single guesthouse result to the response generator. The response is "I have confirmed your booking at Royal Spice with reference [ref]. For accommodation, I found a guesthouse in the east with a moderate price range." Royal Spice is not in the database results given to the response generator, so the booking confirmation is made up.

A non-canonical slot placeholder is when the dialogue state tracker stores a placeholder string like [hotel_name] as the value of a slot, instead of the actual entity name the user mentioned. It shows up in Experiment 1 with the single-LLM setup, where the same prompt that asks the model to extract slots also describes the placeholder vocabulary used in the response, and the model occasionally confuses the two and writes the placeholder token where it should write the value. The number of affected turns is small, but every affected turn loses JGA and Slot F1 because the placeholder string never matches the ground truth value. In dialogue SNG02172 the user says "I choose the ashley hotel. What is their address, please?". The ground truth slot value is hotel-name equal to "ashley hotel", but GPT-4.1-nano writes hotel-name equal to [hotel_name], storing the placeholder token itself in the belief state instead of the entity name the user provided.

A fine-tuning hallucination amplification is when the fine-tuned response generator emits real entity names where the zero-shot version of the same model would have emitted placeholders. It appears only in Experiment 3 and the rates jump sharply. Claude Haiku 4.5 ends Experiment 2 at 0.8 percent hallucination, while the fine-tuned configurations land between 14 and 46 percent: Qwen3-14B at 14 percent, Phi-4-14B at 15.5, Qwen3-8B at 22, LLaMA-3.2-3B at 27.9, and LLaMA-3.1-8B at 45.6. The cause is in the training data. It was generated by string-matching delexicalization on the gold MultiWOZ responses, which only replaced the active domain's entity names with placeholders. On multi-domain turns the non-active domain's names stayed lexicalized in the training response. The fine-tuned models learn this noise as signal and emit real entity names at inference, even on single-domain turns. A second visible signature of the same root cause is format leakage. Phi-4's chat-template token <|im_sep|> appears in some responses, and LLaMA-3.1-8B sometimes emits training-format role markers like "USER:" and "SYSTEM:" mid-response. In dialogue SNG0888 the user says "Four. Two nights. Beginning Saturday.", the database returns the entity "alpha-milton guest house" for the hotel query, but the fine-tuned Phi-4-14B response is "<|im_sep|> I have booked you at the hamilton lodge for 4 people for 2 nights starting Saturday. Your reference number is [ref].", a freshly invented hotel name plus the leaked chat-template token.

Two patterns emerge across the five mechanisms. The dominant source of failure is dialogue state tracking. Multi-domain turn confusion and slot extraction errors together drive Joint Goal Accuracy below half in every configuration of every Experiment, with each small extraction mistake permanently fixed in the cumulative belief state. The second pattern is that hallucination changes shape across the three Experiments rather than disappearing. Experiment 1 shows DB-vs-prompt mismatch from the single LLM ignoring the database in its prompt. Experiment 2 reduces hallucination sharply for the API models because the placeholder rule in the response generator structurally forces it down. Experiment 3 sees the rate climb again, in some cases above forty percent, because the same rule cannot be enforced after fine-tuning. The non-canonical placeholder errors in Experiment 1 and the format leakage in Experiment 3 are surface signs of the same underlying issue: the model has not learned to keep the placeholder vocabulary separate from the actual slot values.

7.6 Limitations

The dataset itself is the first source of unfair penalty in our results. MultiWOZ 2.2 was annotated by humans, and on many turns there is more than one valid answer to the user's request, but only the annotator's choice is treated as ground truth. The pipeline can do exactly what the user asked for and still have its Joint Goal Accuracy fail, because it picked a different but equally valid entity from the database. As a dummy example, the user asks for a moderate hotel in the north of Cambridge, the database returns five hotels matching the request, the pipeline picks the first one, and the annotator originally picked the third. Both choices satisfy the user's request, but only the annotator's choice counts as correct, so JGA fails on that turn even though the pipeline did nothing wrong. A smaller variant of the same issue is multi-value slot lists, where "rosa's" and "rosas bed and breakfast" refer to the same entity but only one is the canonical GT. The combined effect is that no system can reach a perfect JGA or BLEU on this benchmark, and that limits how much we can attribute an imperfect headline number to model weakness rather than to dataset noise.

The training data we used for Experiment 3 carries a similar source of noise that our pipeline created itself. To prepare supervised pairs for fine-tuning, we ran a string-matching delexicalizer over the gold MultiWOZ system responses, replacing entity names with placeholders such as [hotel_name] and [restaurant_name]. The matcher only checks for entity names belonging to the active domain on a given turn. On multi-domain turns (which occur inside 506 of our training dialogues), the non-active domain's entity names slip through and stay as real strings inside the training response. Small fine-tuned models then learn that writing a real entity name in that position is a normal pattern, and at test time they reproduce it. As a dummy example, a training turn covers a hotel called Acorn Guest House (the active domain on this turn) and a restaurant called Saigon City (mentioned earlier but not active on this turn). The delexicalizer turns Acorn Guest House into [hotel_name] but leaves Saigon City untouched. The fine-tuned model then sees many examples where a real restaurant name is acceptable in the response, and at inference time it produces real names instead of [restaurant_name] placeholders. This is the architectural cause of the hallucination amplification described in 7.5, and it limits how much of Experiment 3's hallucination rate can be attributed to the model itself rather than to the training data we fed it.

Our evaluation metrics also come from architectural choices that are simple but cannot capture every case. The hallucination metric uses substring matching: it searches the response for the entity name from the database. The same entity can have more than one valid name form in the data, and we always use the first one as canonical. As a dummy example, the database entity is named "rosas bed and breakfast" but the response says "I found rosa's for you". Both refer to the same hotel, but the substring search for "rosas bed and breakfast" does not find it in the response, so the metric records the turn as having no entity mention at all. The policy violation metric works in a similar way. It flags a turn when the response contains booking-related words such as "reservation" or "your booking" while a booking intent still has missing required slots. The response generator naturally uses such words while asking the user for the missing details, and the metric counts these as violations even when the response is correctly handling the turn. This applies to Experiments 2 and 3, where the booking rules sit in a separate downstream layer rather than inside the LLM's prompt as in Experiment 1. The combined effect is that the hallucination percent and policy violation percent in our results tables do not always reflect the model's true behavior.

8. Conclusion

8.1 Summary of contributions and findings

This work built and evaluated three architectures for task-oriented customer-service dialogue on the hotel and restaurant domains of MultiWOZ 2.2. The first is a single general-purpose LLM that handles the full per-turn pipeline, namely domain and intent classification, slot extraction, entity selection, and response generation, under one monolithic prompt. The second is a zero-shot modular pipeline in which dialogue state tracking and response generation are performed by two separate LLM modules, connected by deterministic database lookup, policy gate, supervisor, lexicalizer, and memory components. The third keeps the modular pipeline of the second architecture and replaces each general-purpose LLM module with a role-specific LoRA adapter fine-tuned on the MultiWOZ training split via QLoRA on the same open-source base model. The three architectures isolate two distinct levers of system design, namely architectural decomposition (Experiment 1 to Experiment 2) and role-specific specialization (Experiment 2 to Experiment 3).

The three architectures were evaluated on the MultiWOZ 2.2 test split filtered to the hotel and restaurant domains (186 dialogues, 1,100 user turns) over twenty-nine running configurations: nine in Experiment 1, thirteen in Experiment 2, and seven in Experiment 3. The configurations cover commercial APIs (GPT-4o-mini, GPT-4.1-nano, Claude Haiku 4.5) and seven open-source models (Qwen2.5-14B, Qwen3-4B, Qwen3-8B, Qwen3-14B, LLaMA-3.2-3B, LLaMA-3.1-8B, Phi-4-14B). The strongest commercial-API configuration, Claude Haiku 4.5 in the zero-shot modular pipeline of Experiment 2, reached Combined 91.35, JGA 45.1, and Slot F1 87.3 at $2.10 over the 1,100-turn evaluation. The strongest open-source configuration, Phi-4-14B fine-tuned in Experiment 3, reached Combined 70.84, JGA 47.0, and Slot F1 88.9 at zero dollars and 3.44 seconds per turn.

The two architectural levers behave differently. Decomposition is primarily a cost-and-latency lever. API costs drop by approximately 80% to 90% from Experiment 1 to Experiment 2 because the per-role prompts are much shorter than the monolithic prompt, with Claude Haiku 4.5 falling from $21.03 to $2.10 over the same 1,100 turns, and open-source per-turn latency falls three to fivefold for the same reason. The quality effect of decomposition depends on the model. Weaker single-LLM configurations gain substantially, with LLaMA-3.1-8B rising by 36.6 Combined points and GPT-4o-mini by 24.2, stronger ones lose ground, with Qwen3-14B falling by 4.8 and Phi-4-14B by 12.1, and Claude Haiku 4.5 stays almost flat at the open-source-unreachable ceiling. Specialization is primarily a dialogue-state-tracking lever. Fine-tuned 14B-class adapters reach JGA and Slot F1 above the Claude Haiku 4.5 zero-shot baseline, while end-to-end Combined remains capped roughly twenty points below Claude Haiku 4.5 because response generation does not close that gap. Within the modular architecture itself, dialogue state tracking is the dominant of the two LLM roles. Swapping Claude Haiku 4.5 for GPT-4o-mini in the dialogue state tracker hurts the system about three times more than the same swap in the response generator, and a stronger dialogue state tracker can recover most of the loss that decomposition caused on a strong model that decomposition hurt.

Two more findings come from comparing the three Experiments side by side. The first is that the property predicting whether fine-tuning helps a given model is not its parameter count but how well the same model already followed the placeholder convention in Experiment 2. Models that broke that convention in Experiment 2 (increased hallucination rate), such as LLaMA-3.2-3B, Phi-4-14B, and Qwen3-8B, gain substantially from fine-tuning, while models that already followed it, such as Qwen3-14B and Qwen3-4B, lose ground. The Phi-4-14B and Qwen3-4B contrast is the clearest case, where the larger Phi-4-14B was off-convention in Experiment 2 and ends up the strongest 14B fine-tune, while the smaller Qwen3-4B was on-convention and loses the most. The second is that the gap to commercial APIs in Experiment 3 is shaped mostly by the training data we built rather than by the modular architecture. The supervised pairs used to fine-tune the response generator were produced by a string-matching delexicalizer over the MultiWOZ 2.2 system responses. On multi-domain turns the rule only replaces active-domain entity names, imperfect string matching leaves further entity strings unchanged, and a small number of MultiWOZ ground-truth responses are themselves damaged. The resulting training signal teaches the model that real entity names are sometimes acceptable in positions where the placeholder convention requires a token, which shows up at inference as elevated hallucination, especially in the smaller and LLaMA-family models. Better delexicalization rules, alias-aware matching, and a quick review of the ground-truth responses would each remove part of this signal, with no change to the pipeline structure or the fine-tuning recipe.

8.2 Future Work

The first direction is to clean up the training data we built ourselves for Experiment 3. The supervised pairs used to fine-tune the response generator were produced by a string-matching rule that replaces entity names with placeholders such as [hotel_name] and [restaurant_name]. In 8.1 we traced the gap to commercial APIs to three small problems in this rule. On multi-domain turns the rule only replaces the active domain's names and leaves the rest of the entity strings untouched. The string matching itself misses some name variants. And a small number of the original MultiWOZ system responses are themselves damaged. Each of these leaks a bit of bad signal into training, and small models, especially those in the LLaMA family, learn that real entity names are sometimes acceptable in positions where a placeholder is expected. Fixing all three is cheap. The same fine-tuning recipe would then reach a higher score with no change to the pipeline structure.

The second direction is to switch the dialogue state tracker from a JSON-string protocol to a tool-calling protocol. In the present pipeline the model writes its slot values as a JSON string inside its text reply, and our parser extracts the JSON afterwards. Sometimes the model adds extra words around the JSON or writes the wrong format, and the turn fails. In tool calling we instead give the model a function definition such as book_hotel(area, stars, ...), and the model fills the slot values directly as named arguments to that function. We receive the slot values directly as a Python dictionary, with no parsing needed. In 8.1 we identified the dialogue state tracker as the dominant of the two LLM roles in the modular pipeline, so a cleaner output format here has the biggest payoff.

The third direction is to extend the evaluation from the hotel and restaurant domains used here to the full set of seven MultiWOZ 2.2 domains, which also includes attraction, taxi, train, hospital, and police. The two-domain choice is the main scope limit of the present work. It also keeps our numbers from being compared one-to-one with the published MultiWOZ leaderboard, since most of the literature reports the standardized metrics over all seven domains. Extending the dataset filters and the policy gate is mostly engineering work, since the rest of the pipeline is already domain-agnostic by design. The result would put this work on the same map as the published baselines.

The fourth direction is to add retrieval-augmented few-shot prompting as a third architectural alternative next to zero-shot prompting (Experiment 2) and fine-tuning (Experiment 3). We would build a vector index over the MultiWOZ training set once, and at runtime a retriever pulls the two to five training dialogues most similar to the current user input and injects them into the dialogue state tracker and response generator prompts as in-context examples. No fine-tuning is needed and no architecture change is needed. The model simply sees concrete examples of correct slot extraction and correct response wording for the kind of input it just received. This goes beyond the example-based prompting we already use. Our current prompts contain a handful of hand-written instruction examples added during development to address specific errors we observed in model behavior, and those examples are fixed and shared across every user input. A retriever-based version would instead pick fresh examples for each turn from the training set. Hudecek and Dušek [10] reported joint goal accuracy rising from about 30 zero-shot to about 37 few-shot on MultiWOZ 2.2, so the direction is grounded in published work and tightly tied to our setup. The cost is a slightly larger prompt per call and the engineering of an index plus a retriever, both of which are cheap.

The fifth direction is to write a different prompt for each model. In the present work we used the same dialogue state tracker prompt and the same response generator prompt for every model in every Experiment, so that performance differences could be tied to the models themselves rather than to prompt engineering. Future work could change the prompt per model, since one model often performs better with one phrasing while another performs better with a different one. The smaller open-source models would likely gain the most, since under the identical-prompt setup they sometimes fail on instructions that the commercial APIs follow easily.

One more direction is to add a new Experiment 4 on top of Experiment 3. Experiment 3 fine-tunes the LoRA adapters with supervised learning, where the model is shown the correct response for each training input and learns to copy it. Experiment 4 would add another round of fine-tuning on top, this time with preference learning, using a method called DPO (Direct Preference Optimization). DPO works on a simple recipe. We give it many pairs of two candidate responses to the same input, each pair labeled with which response is better. The training then teaches the model to prefer the chosen one over the rejected one. No reward model and no reinforcement learning are needed. The preference pairs themselves can be made cheaply by having the Experiment 3 model produce the rejected response and a stronger commercial model produce the preferred response on the same training input. The target is the response-generator gap to commercial APIs that supervised fine-tuning alone did not close. DPO has been used on task-oriented dialogue with reported gains over standard fine-tuning [16], and it plugs into the LoRA setup of Experiment 3 with little extra engineering.

The last direction is more speculative and points outside the academic focus of the present work, toward a potential production extension. The pipeline of Experiments 2 and 3 runs the same linear sequence on every turn: dialogue state tracker, database, response generator. A real customer-service product also has to deal with paths that break this linear shape. The customer can want to book, change a booking, cancel, complain, or hand off to a human. A natural extension is to place a workflow graph on top of the pipeline, with one node per path. The graph routes each turn to the relevant node, and the modular pipeline runs inside each node as we already have it. This is the most direct route from the modular pipeline of this work to a deployable customer-service product.

References

[1] P. Budzianowski, T.-H. Wen, B.-H. Tseng, I. Casanueva, S. Ultes, O. Ramadan, M. Gasic, “MultiWOZ - A Large-Scale Multi-Domain Wizard-of-Oz Dataset for Task-Oriented Dialogue Modelling”, in *EMNLP, p. 5016-5026,* 2018.

[2] X. Zang, A. Rastogi, S. Sunkara, R. Gupta, J. Zhang, J. Chen, “MultiWOZ 2.2: A Dialogue Dataset with Additional Annotation Corrections and State Tracking Baselines”, in *NLP4ConvAI, p. 109-117,* 2020.

[3] E. Hosseini-Asl, B. McCann, C.-S. Wu, S. Yavuz, R. Socher, “A Simple Language Model for Task-Oriented Dialogue”, in *NeurIPS,* 2020.

[4] T. B. Brown, B. Mann, N. Ryder, et al., “Language Models are Few-Shot Learners”, in *NeurIPS,* 2020.

[5] L. Ouyang, J. Wu, X. Jiang, et al., “Training Language Models to Follow Instructions with Human Feedback”, in *NeurIPS,* 2022.

[6] H. Touvron, T. Lavril, G. Izacard, et al., “LLaMA: Open and Efficient Foundation Language Models”, *arXiv preprint arXiv:2302.13971,* 2023.

[7] A. Yang, B. Yang, B. Hui, et al., “Qwen2.5 Technical Report”, *arXiv preprint arXiv:2412.15115,* 2024.

[8] A. Yang, B. Yang, B. Hui, et al., “Qwen3 Technical Report”, *arXiv preprint arXiv:2505.09388,* 2025.

[9] M. Abdin, J. Aneja, H. Behl, et al., "Phi-4 Technical Report", *arXiv preprint arXiv:2412.08905,* 2024.

[10] V. Hudecek, O. Dušek, “Are Large Language Models All You Need for Task-Oriented Dialogue?”, in *SIGDIAL p. 216-228,* 2023.

[11] X. Zhang, R. Peng, K. Li, J. Zhou, H. Meng, “SGP-TOD: Building Task Bots Effortlessly via Schema-Guided LLM Prompting”, in *Findings of EMNLP, p. 13348-13369,* 2023.

[12] Y. Hu, C.-H. Lee, T. Xie, et al., “In-Context Learning for Few-Shot Dialogue State Tracking”, in *Findings of EMNLP, p. 2627-2643,* 2022.

[13] H. Xu, X. Mao, P. Yang, F. Sun, H. Huang, “Rethinking Task-Oriented Dialogue Systems: From Complex Modularity to Zero-Shot Autonomous Agent”, in *ACL, p. 2748-2763,* 2024.

[14] A. Gupta, A. Ravichandran, Z. Zhang, S. Shah, A. Beniwal, “DARD: A Multi-Agent Approach for Task-Oriented Dialog Systems”, *arXiv preprint arXiv: 2411.00427,* 2024.

[15] A. Baidya, K. Das, X. Gao, “The Behavior Gap: Evaluating Zero-shot LLM Agents in Complex Task-Oriented Dialogs”, *arXiv preprint arXiv: 2506.12266,* 2025.

[16] Z. Feng, X. Wang, B. Wu, W. Zhong, Z. Xu, H. Cao, T. Zhao, Y. Li, B. Wang, “Empowering LLMs in Task-Oriented Dialogues: A Domain-Independent Multi-Agent Framework and Fine-Tuning Strategy”, *arXiv preprint arXiv: 2505.14299,* 2025.

[17] C. Kranti, S. Hakimov, D. Schlangen, “clem:todd: A Framework for the Systematic Benchmarking of LLM-Based Task-Oriented Dialogue System Realisations”, *arXiv preprint arXiv:2505.05445,* 2025.

[18] J. Wei, X. Wang, D. Schuurmans, M. Bosma, B. Ichter, F. Xia, E. Chi, Q. Le, D. Zhou, “Chain-of-Thought Prompting Elicits Reasoning in Large Language Models”, in *NeurIPS,* 2022.

[19] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, W. Chen, “LoRA: Low-Rank Adaptation of Large Language Models”, in *ICLR,* 2021.

[20] T. Dettmers, A. Pagnoni, A. Holtzman, L. Zettlemoyer, “QLoRA: Efficient Finetuning of Quantized LLMs”, in *NeurIPS,* 2023.

[21] Q.-V. Nguyen, Q.-C. Nguyen, H. Pham, K.-H. N. Bui, “Spec-TOD: A Specialized Instruction-Tuned LLM Framework for Efficient Task-Oriented Dialogue Systems”, in *SIGDIAL,* 2025.

[22] J. Deriu, A. Rodrigo, A. Otegi, G. Echegoyen, S. Rosset, E. Agirre, M. Cieliebak, “Survey on evaluation methods for dialogue systems”, *Artificial Intelligence Review, vol. 54, p. 755–810,* 2021.

[23] T. Nekvinda, O. Dušek, “Shades of BLEU, Flavours of Success: The Case of MultiWOZ”, in *GEM, p. 34-46,* 2021.